INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] DevOps and Security

LS lab 5 Report

On

Using Large Language Models (LLMs) in DevOps Pipelines

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 23.02.2026

Index

Introduction……...3

Task 1: CI/CD Pipeline Baseline with Security and Kubernetes……...4-12

Task 2: LLM Setup and Deployment……..13-18

Task 3: Testing and Validation……19-22

Task 4: Security and Trust Analysis……23-28

Bonus Task: Advanced Use of Large Language Models in DevOps……...30-41

References……..41-42

Introduction:

In this report, I implemented a full DevOps lab based on three VMs using Ubuntu on VMware, where GitLab (*http://gitlab-24.sne.com:8090*) was used to automate everything from source code to build, test, security scanning, container image creation and push to registry, and finally Kubernetes (*Minikube* on VM3) (*deploy-24*). VM1 was mainly used for GitLab Server/UI and evidence collection, VM2 (*runner-24*) was used for repository changes, pipeline configuration (*.gitlab-ci.yml*), local LLM API test and commit/push to trigger the pipeline, and VM3 (*deploy-24*) was used to set up the *Minikube* cluster and *kubectl* access and execute the Kubernetes deployment stage. To strengthen the DevSecOps baseline, I added a *SAST* stage with Bandit after build/test, so that potential security issues in the code are caught in advance and the report is saved as an artifact.

Then, in the "*LLM Setup and Deployment*" section, I used the DeepSeek Chat API as an external LLM API to collect the API key, store it securely as a GitLab CI/CD variable, and test it locally in VM2 with a simple prompt, ensuring that the LLM call from the pipeline would actually work. I added a new "*llm-review*" stage to the pipeline to send the source code or pipeline configuration to LLM for analysis, and saved the output as a *JSON* artifact, so that each pipeline run automatically provides a readable summary and improvement suggestions. By placing the LLM stage after build and test, I ensured that it would pass the deterministic quality gate (*build/test/sast*) first, and then the assistive analysis (LLM) would run, reducing resource waste due to misleading suggestions on incorrect or unstable code and unnecessary external API dependencies. In the Testing & Validation section, I verified that the pipeline triggered the LLM job successfully using the job log and artifact, and observed whether the LLM output adequately explained what the application did. In addition, in the Security & Trust Analysis section, we identified the risks of sending *source/log/config* to external LLM (IP leakage, secrets/PII exposure, prompt injection, hallucination, vendor retention policy) and critically evaluated the valid vs. questionable/incorrect points of the output by analyzing the same pipeline through security, reliability, and trust lenses using refined prompt. Finally, in the bonus section, we demonstrated deploying an extra local lightweight LLM (*Ollama + phi-3*), and further advanced the pipeline with failed job logs analysis, outdated dependency detection, *README* improvement suggestion, etc.

Task 1: CI/CD Pipeline Baseline with Security and Kubernetes

Before introducing the LLM, you will prepare a secure DevOps baseline.

1- Show the CI/CD pipeline you implemented in Lab3, including the following stages:

- **Source code checkout**
- **Build**
- **Test**
- **Container image build and deploy**

2- Add a Static Application Security Testing (SAST) stage to your pipeline (using tools like Bandit, Semgrep, Sonarqube). Show the output of the security scan

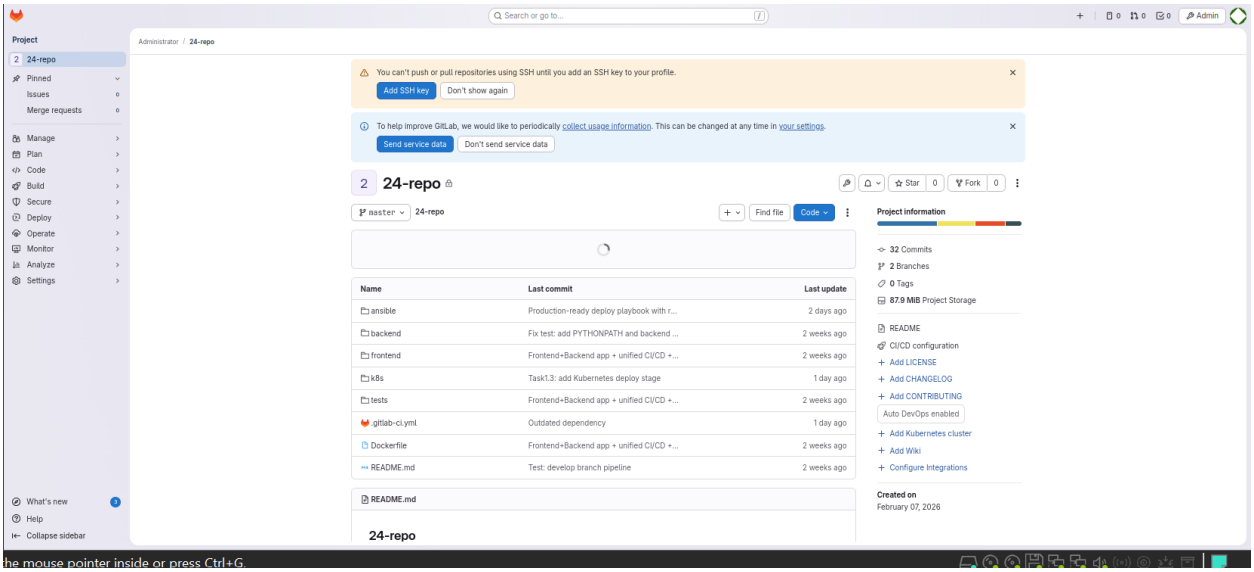
3- Extend your pipeline to include a deployment stage to Kubernetes.

Solution:

Task 1.1

In Lab3, I created an automated pipeline using GitLab CI/CD, where *build, test, docker_build & push, deploy_ansible* are completed sequentially as the code is pushed.

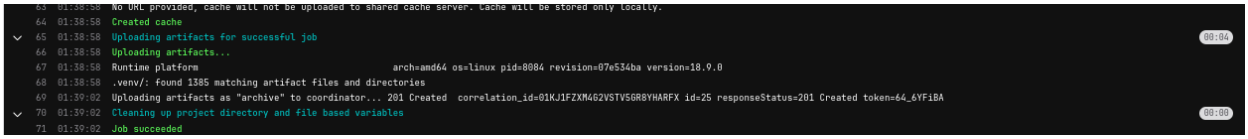
When the pipeline starts, GitLab Runner automatically clones the source code from the repository.



Build Stage

In this stage, I installed dependencies and built the application. If the build is successful, it goes to the next stage.

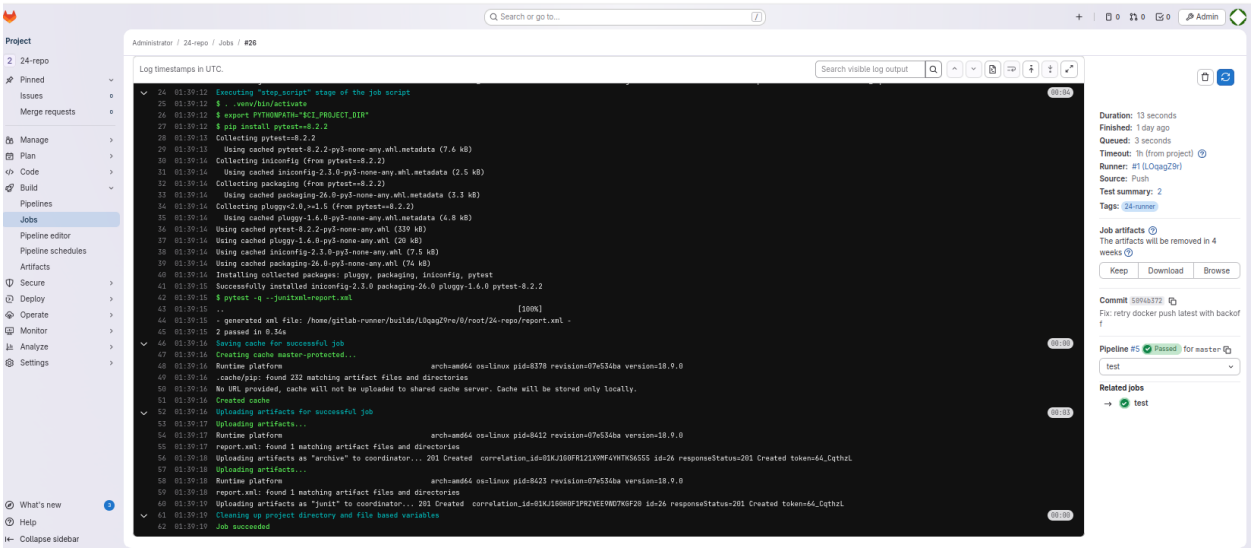
Evidence: "*Job succeeded*" in the build job log.



Test Stage

After the build, I ran the test so that if there is an error, the pipeline stops here.

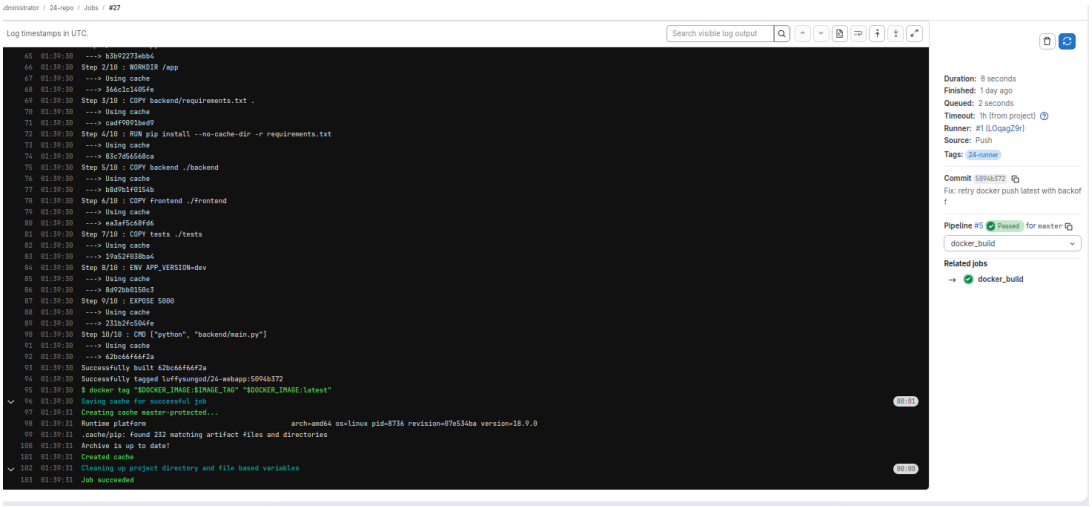
Evidence: The test job has completed successfully.



Docker Build & Push

After passing the test, I built the image using *Dockerfile* and pushed it to *DockerHub*.

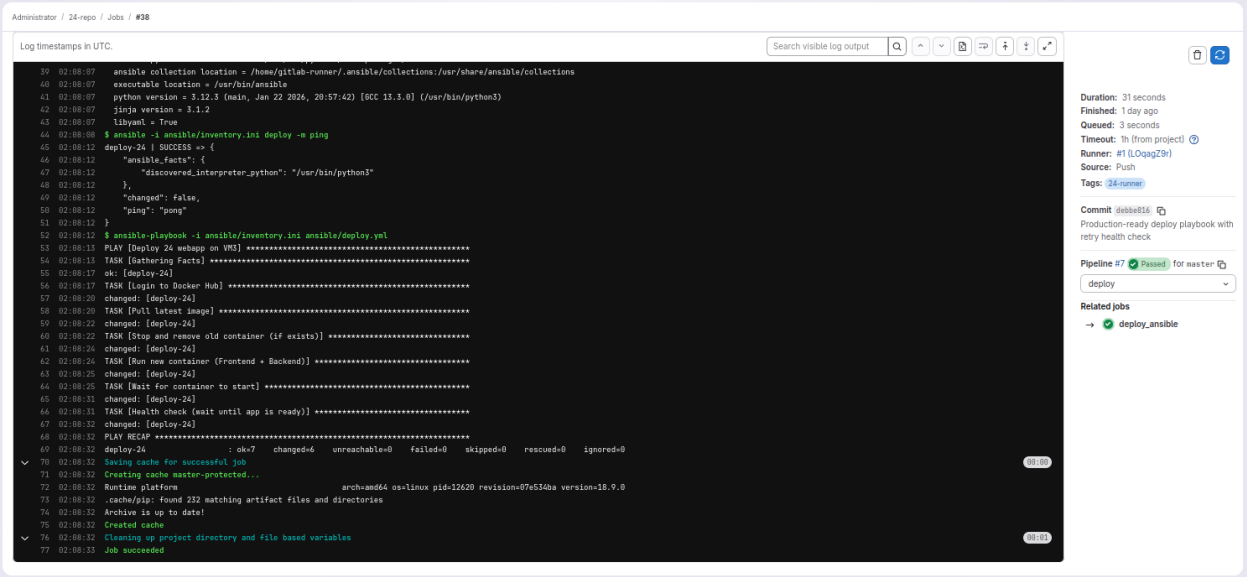
Evidence: *docker_build* and *docker_push* are successful in the job log and a new image tag is visible in *DockerHub.*



Deploy_ansible Stage

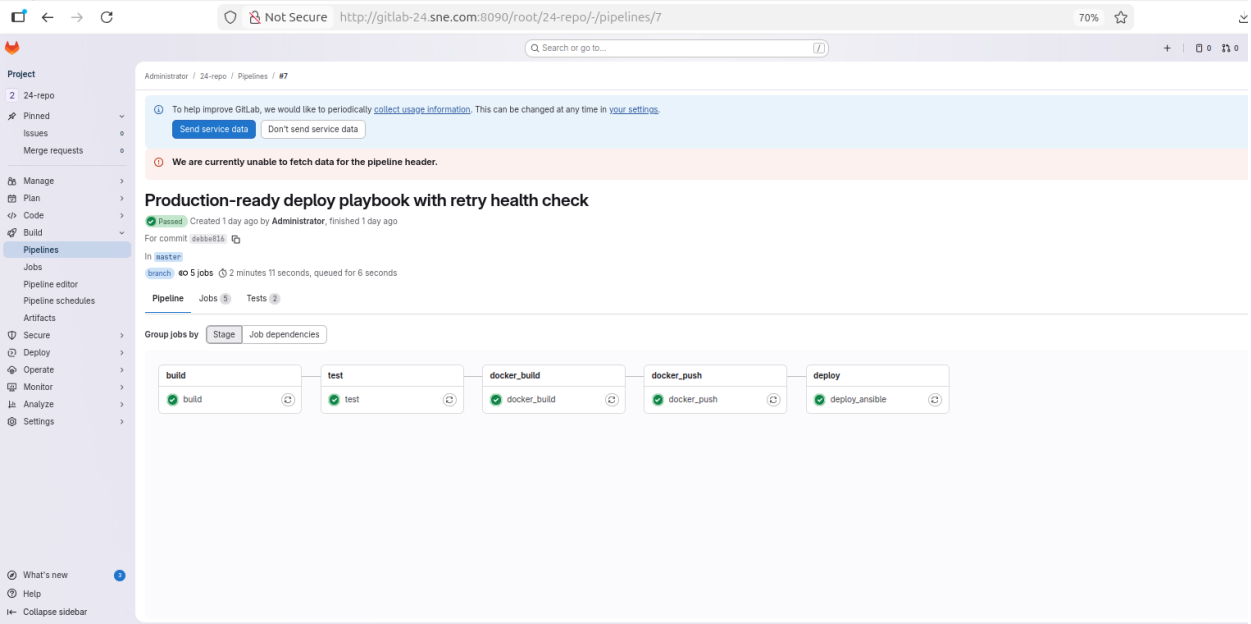
Finally, in the *Deploy_ansible* stage, I deployed the application to the VM3/Kubernetes environment using the new image.

Evidence: *Deploy_ansible* job success and container running on target server.



Proof:

Going to GitLab on VM1, from CI/CD to Pipelines, the stage list clearly shows *build / test / docker / deploy* and the entire pipeline is green.



Task 1.2

In this task, I added a Static Application Security Testing (*SAST*) stage to the CI/CD pipeline, where I used Bandit to analyze the security vulnerabilities of the Python application. Initially, due to the externally-managed-environment (*PEP668*) error, I decided to run Bandit by creating a virtual environment (*venv*) instead of using system Python directly, so that the runner environment is secure and there are no dependency conflicts.

First, I open a terminal in VM2 and go to my project directory *(~/24-repo)*.

```
hasib@runner-24:~$ cd ~/24-repo
```

Then edit the *.gitlab-ci.yml* file.

```
(.venv_ds) hasib@runner-24:~/24-repo$ nano .gitlab-ci.yml
```

There, add the *sast* stage to the stages section, so that security scanning can be run after build and test.

```
stages:
  - build
  - test
  - sast
  - docker_build
  - docker_push
  - deploy
```

Then I define a *sast* job where first the Python version is verified, then a virtual environment is created with *python3 -m venv .venv_sast* . Activate that environment and *run pip install --upgrade pip* and *pip install bandit*. Finally I run a security scan on the backend source code using the *bandit -r backend -f json -o bandit-report.json* command and save the report in JSON format. To make the report always available, I attach it as an artifact (*when: always*) and keep it for 14 days.

```
sast:
  stage: sast
  script:
    - python3 --version
    - python3 -m venv .venv_sast
    - . .venv_sast/bin/activate
    - pip install --upgrade pip
    - pip install bandit
    - bandit -r backend -f json -o bandit-report.json || true
  artifacts:
    when: always
    paths:
      - bandit-report.json
    expire_in: 14 days
  tags:
    - "24-runner"
```

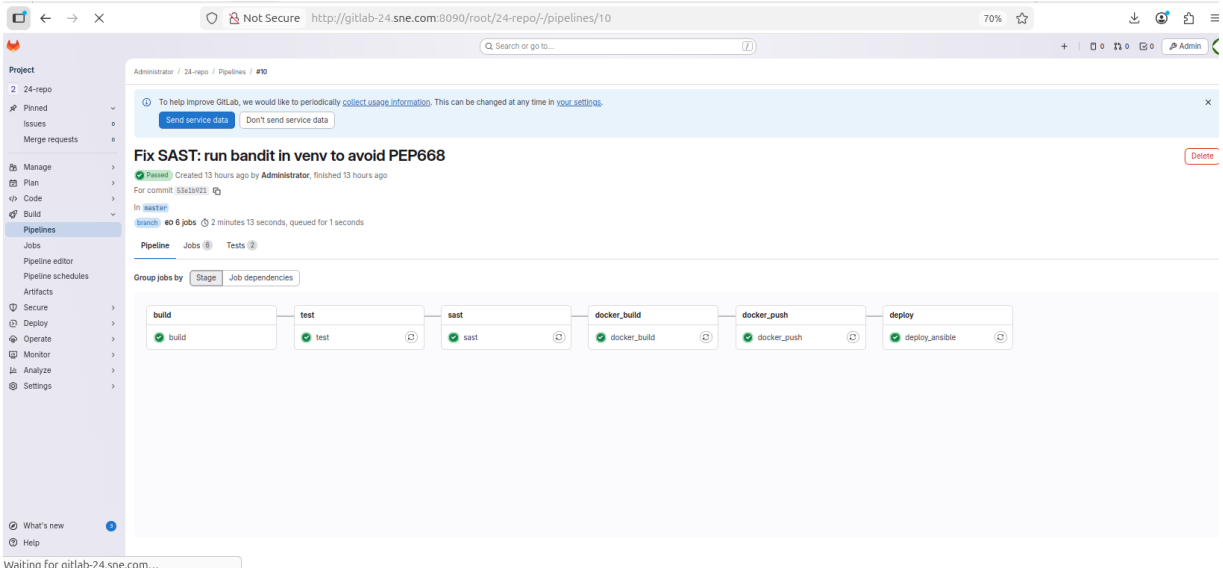
I save the file, commit and push, which automatically triggers the pipeline.

```
(.venv_ds) hasib@runner-24:~/24-repo$ git add .gitlab-ci.yml
```

```
(.venv_ds) hasib@runner-24:~/24-repo$ git commit -m "Fix SAST:run bandit in venv to avoid PEP668"
On branch master
Your branch is up to date with 'origin/master'.
```

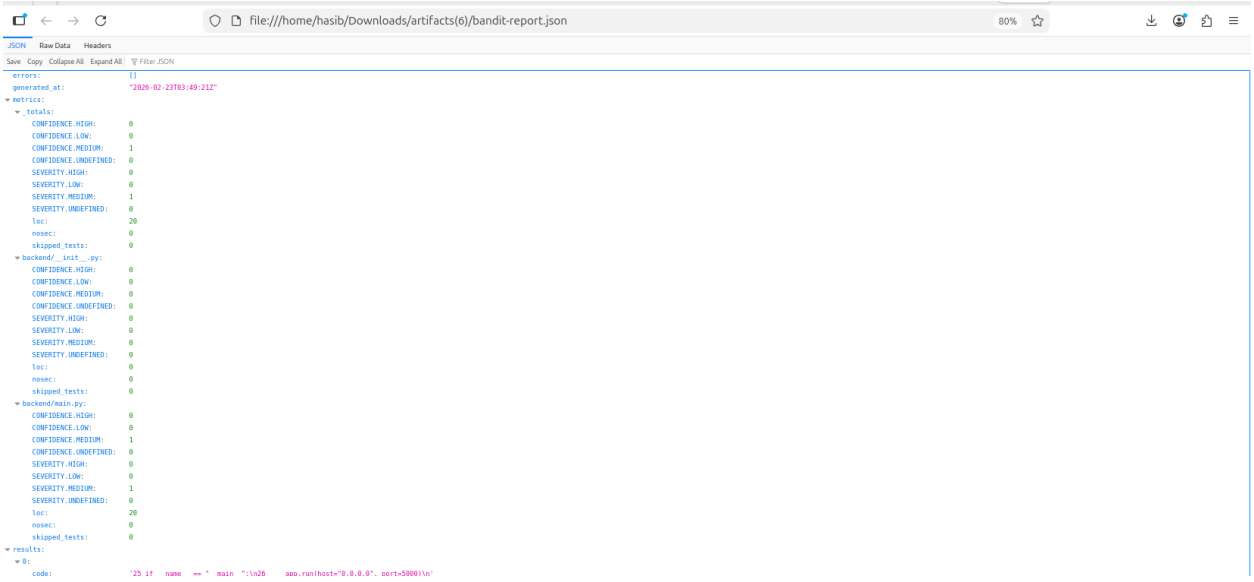
```
(.venv_ds) hasib@runner-24:~/24-repo$ git push
Username for 'http://gitlab-24.sne.com:8090': root
Password for 'http://root@gitlab-24.sne.com:8090':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 577 bytes | 192.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To http://gitlab-24.sne.com:8090/root/24-repo.git
   55c0270..d3089d5  master -> master
```

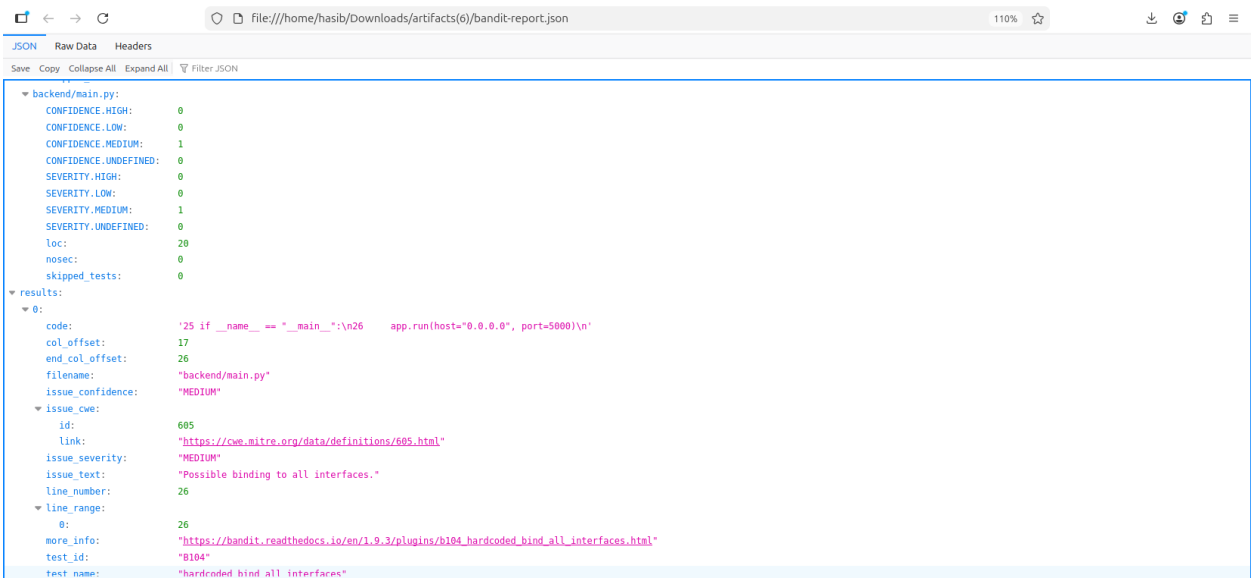
On VM1, go to the GitLab UI and open the latest pipeline from Project to CI/CD to Pipelines to see that the new *sast* stage has run successfully.



I downloaded and opened *bandit-report.json* from the Artifacts section to verify the scan output.

opening screenshot of *bandit-report.json* from Artifacts





Task 1.3

To extend the Lab3 pipeline to Kubernetes, I added a deploy stage using a *Minikube* Kubernetes cluster on VM3. First, I verified that the *Minikube* node is showing Ready in the kubectl get nodes command on VM3.

```
hasib@deploy-24:~$ sudo -u gitlab-runner kubectl get nodes
[sudo] password for hasib:
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   12h   v1.35.1
```

But while running the deploy job, I was getting permission denied issue on *.minikube/.kube* files for *gitlab-runner* user. To solve this issue, I created a separate */home/gitlab-runner/.kube* directory for *gitlab-runner* in VM3, then copied the */home/hasib/.kube/config* file there and fixed the ownership and permission.

```
hasib@deploy-24:~$ sudo mkdir -p /home/gitlab-runner/.kube
```

```
hasib@deploy-24:~$ sudo cp /home/hasib/.kube/config /home/gitlab-runner/.kube/config
```

```
hasib@deploy-24:~$ sudo chown -R gitlab-runner:gitlab-runner /home/gitlab-runner/.kube
sudo chmod 600 /home/gitlab-runner/.kube/config
```

Also, to read *Minikube*'s cert/key file, use *setfacl* on the */home/hasib/.minikube* and */home/hasib/.kube* paths to give *gitlab-runner read/execute* access.

```
hasib@deploy-24:~$ sudo setfacl -R -m u:gitlab-runner:rx /home/hasib/.minikube
```

```
hasib@deploy-24:~$ sudo setfacl -R -m u:gitlab-runner:rx /home/hasib/.kube
```

Then, I have run the following two commands to ensure that the *gitlab-runner* user can successfully access the Kubernetes cluster and that cluster-info is coming in correctly.

```
hasib@deploy-24:~$ sudo -u gitlab-runner kubectl get nodes
[sudo] password for hasib:
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   12h   v1.35.1
```

```
hasib@deploy-24:~$ sudo -u gitlab-runner kubectl cluster-info
Kubernetes control plane is running at https://192.168.49.2:8443
CoreDNS is running at https://192.168.49.2:8443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Then, in VM2, I create a folder called k8s inside my repo *(~/24-repo*) and create a *k8s/deployment.yaml* manifest, where I define a Deployment and a *NodePort* Service.

```
hasib@runner-24:~$ cd ~/24-repo
```

I create a virtual environment (*venv*) and use the image *(MY_DOCKERHUB/24-webapp:latest)* pushed to my *DockerHub* in the *image: field*.

```
hasib@runner-24:~/24-repo$ python3 -m venv .venv_ds
. .venv_ds/bin/activate
```

```
(.venv_ds) hasib@runner-24:~/24-repo$ mkdir -p k8s
```

The main purpose of creating this manifest was to deploy the application by running *kubectl apply* directly from the pipeline.

```
(.venv_ds) hasib@runner-24:~/24-repo$ nano k8s/deployment.yaml
```

```
 GNU nano 7.2                                          k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-24
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp-24
  template:
    metadata:
      labels:
        app: webapp-24
    spec:
      containers:
        - name: webapp-24
          image: luffysungod/24-webapp:latest
          ports:
            - containerPort: 5000
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: webapp-24-service
spec:
  type: NodePort
  selector:
    app: webapp-24
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30007

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify
```

Then I confirm the deploy stage in the stages section of the *.gitlab-ci.yml* file and add a *deploy_k8s* job, which will run only on the master branch.

```
(.venv_ds) hasib@runner-24:~/24-repo$ nano .gitlab-ci.yml
```

```
  GNU nano 7.2
stages:
  - build
  - test
  - sast
  - docker_build
  - docker_push
  - deploy
```

In this job, we verify the client with *kubectl version --client*, then run *kubectl apply -f k8s/deployment.yaml --validate=false* (because there was an OpenAPI validation download error earlier), and finally confirm whether the deployment and service are running with *kubectl get deploy,svc*.

```
deploy_k8s:
  stage: deploy
  script:
    - export KUBECONFIG=/home/gitlab-runner/.kube/config
    - kubectl get nodes
    - kubectl apply -f k8s/deployment.yaml
    - kubectl get deploy,svc
  tags: ["k8s-runner"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "master"'
```

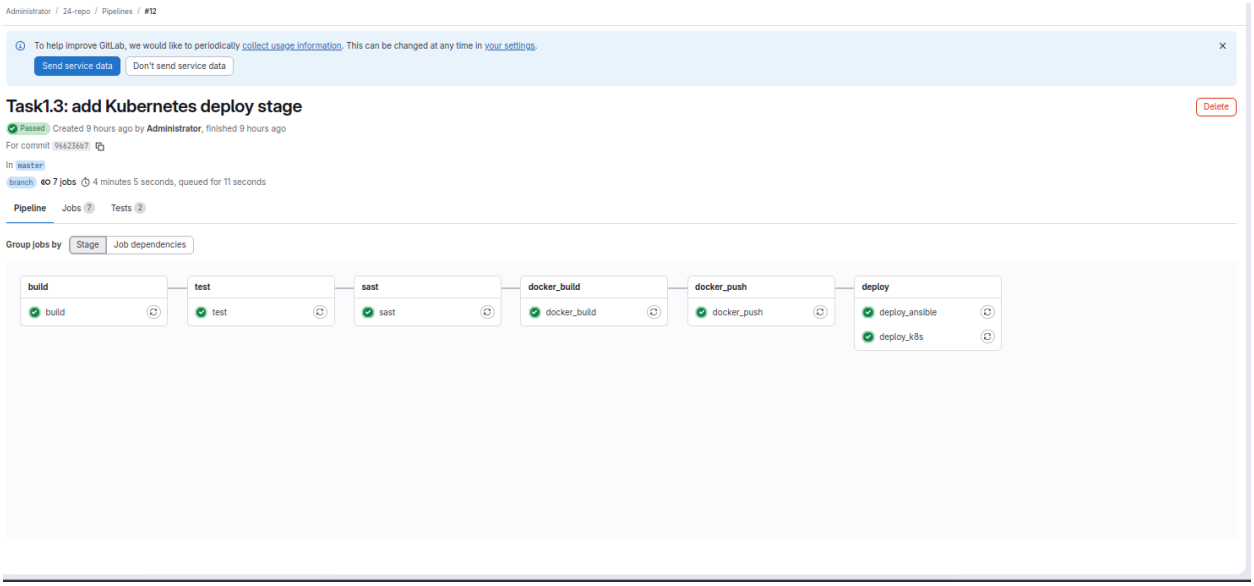
Finally, I add *k8s/deployment.yaml* and *.gitlab-ci.yml,* commit and push, resulting in the pipeline trigger and the *deploy_k8s* job running successfully.

```
(.venv_ds) hasib@runner-24:~/24-repo$ git add k8s/deployment.yaml .gitlab-ci.yml
```

```
(.venv_ds) hasib@runner-24:~/24-repo$ git commit -m "Task1.3: add Kubernetes deployment stage"
On branch master
Your branch is up to date with 'origin/master'.
```

```
(.venv_ds) hasib@runner-24:~/24-repo$ git push
Username for 'http://gitlab-24.sne.com:8090': root
Password for 'http://root@gitlab-24.sne.com:8090':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.22 KiB | 624.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To http://gitlab-24.sne.com:8090/root/24-repo.git
   827eca4..55c0270  master -> master
```

I opened the *deploy_k8s* job log from the GitLab UI of VM1, Pipeline, and took screenshots of the *kubectl apply* and *kubectl get deploy*, svc output as evidence, and if necessary, I also verified the running URL by running *minikube* service *webapp-24-service --url on* VM3.



Task 2: LLM Setup and Deployment

In this task, you will prepare an LLM that will later be used in the CI/CD pipeline.

1- Choose an external LLM API (open AI chat API, Gemini API, YAndex GPT API), show how you obtained an API key and tested it locally using a simple prompt.

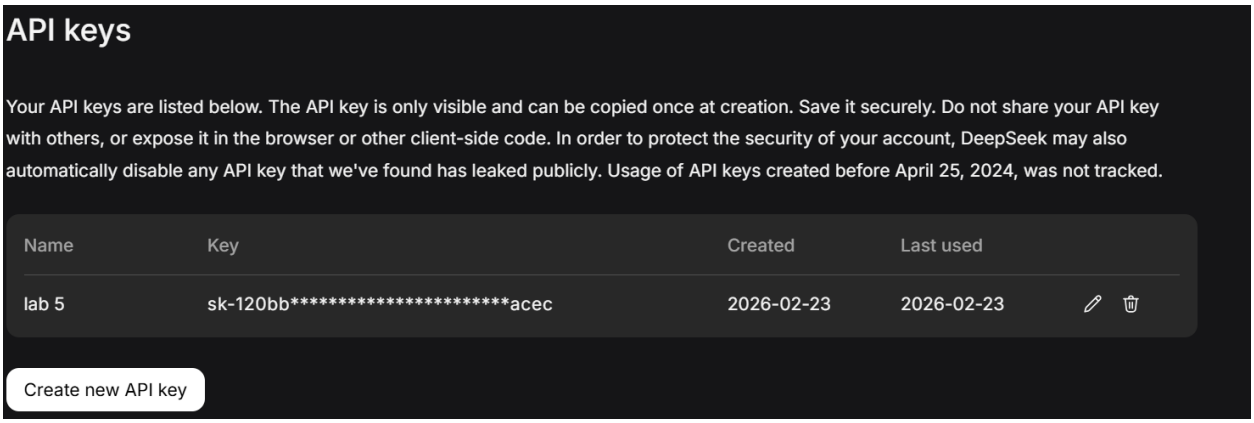
2- Add a new stage called *llm-review* to your existing CI/CD pipeline that will send application source code or pipeline configuration to the LLM and store the response as a pipeline artifact. Then Show the updated pipeline stage.

3- Why should the LLM stage be placed after the build and test stages ?

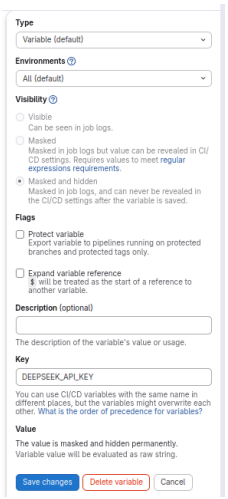
Solution:

Task 2.1

Here, I have selected *DeepSeek* Chat API (*model: deepseek-chat*) as an external LLM API. First, log in to the DeepSeek platform using a browser, generate an API key from the API section and store it securely, as this key will be used to call LLM from the CI/CD pipeline later.



After getting the key, I went to Settings from VM1 (GitLab UI) to Project, went to CI/CD, went to Variables, added a variable called *DEEPSEEK_API_KEY*, where I set the key value and enabled Masked and Hidden for security (I didn't give Protected, because if the master branch is not protected, the job variable won't be accessible).



Then, to verify that the key is working properly, I test the API with a simple prompt locally on VM2.

```
hasib@runner-24:~/24-repo$ python3 -m venv .venv_ds
. .venv_ds/bin/activate
```

For this, I create a virtual environment and activate it (*python3 -m venv .venv_ds* and *. .venv_ds/bin/activate*), then install the requests library.

```
(.venv_ds) hasib@runner-24:~/24-repo$ pip install requests
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Using cached charset_normalizer-3.4.4-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (37 kB)
Collecting idna<4,>=2.5 (from requests)
  Using cached idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Using cached urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2017.4.17 (from requests)
  Using cached certifi-2026.1.4-py3-none-any.whl.metadata (2.5 kB)
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Using cached certifi-2026.1.4-py3-none-any.whl (152 kB)
Using cached charset_normalizer-3.4.4-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (153 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Using cached urllib3-2.6.3-py3-none-any.whl (131 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.1.4 charset_normalizer-3.4.4 idna-3.11 requests-2.32.5 urllib3-2.6.3
```

Then we set *export DEEPSEEK_API_KEY="PASTE_KEY"* as the environment variable and run the *deepseek_test.py* script, which checks the API response by sending a short prompt.

```
(.venv_ds) hasib@runner-24:~/24-repo$ export DEEPSEEK_API_KEY="sk-500272895e3643cb9b21eab5924c42e0"
```

After running, the terminal shows *HTTP Status 200,* which proves that the API key is valid and the LLM endpoint is responding properly. As evidence, I have saved a screenshot of the terminal output (with Status *200*) of VM2, so that the report can show that the API key was successfully obtained and tested effectively with a simple prompt locally.

```
(.venv_ds) hasib@runner-24:~/24-repo$ python3 deepseek_test.py
Status: 200
{"id":"70dfb600-ecea-45de-b8ad-c38c06bd764f","object":"chat.completion","created":1771802226,"model":"deepseek-chat","choices":[{"index":0,"message":{"role":"assistant","content":"CI/CD automates software delivery by continuously integrating code changes and deploying them to production. This ensures faster, more reliable releases with fewer manual errors."},"logprobs":null,"finish_reason":"stop"}],"usage":{"prompt_tokens":12,"completion_tokens":29,"total_tokens":41,"prompt_tokens_details":{"cached_tokens":0},"prompt_cache_hit_tokens":0,"prompt_cache_miss_tokens":12},"system_fingerprint":"fp_eaab8d114b_prod0820_fp8_kvcache"}
```

Task 2.2

In this step, I added a new stage to my existing CI/CD pipeline called *llm-review*. The purpose was to send a specific part of the application source code or pipeline configuration to LLM during the pipeline for analysis/interpretation and save that response as a pipeline artifact. For this, I edited the *.gitlab-ci.yml* file in VM2 and added *llm-review* to the stages: list, so that this stage runs after build, test, *sast,* deploy are completed.

```
(.venv_ds) hasib@runner-24:~/24-repo$ nano .gitlab-ci.yml
```

```
  GNU nano 7.2                                                    .g
stages:
  - build
  - test
  - sast
  - docker_build
  - docker_push
  - deploy
  - llm-review

variables:
  IMAGE_TAG: "$CI_COMMIT_SHORT_SHA"
  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"


cache:
  key: "$CI_COMMIT_REF_SLUG"
  paths:
    - .cache/pip

build:
  stage: build
  script:
    - python3 --version
    - python3 -m venv .venv
    - . .venv/bin/activate
    - pip install --upgrade pip
    - pip install -r backend/requirements.txt
```

Then I add a job called *llm_review*, where to safely avoid PEP668 issues, I create and activate the virtual environment inside the job with *python3 -m venv .venv_llm* and install the requests library.

```
llm_review:
  stage: llm-review
  script:
    - python3 -m venv .venv_llm
    - . .venv_llm/bin/activate
    - pip install --upgrade pip
    - pip install requests
    - |
      python3 - <<'PY'
import os, json, requests

api_key = os.getenv("DEEPSEEK_API_KEY")
code = open("backend/main.py","r",encoding="utf-8",errors="ignore").read()

prompt = f"""You are a DevOps assistant.
Explain what this application does in simple terms.
List 5 improvements.

CODE:
{code[:12000]}
"""
```

```
r = requests.post(
  "https://api.deepseek.com/chat/completions",
  headers={"Authorization": f"Bearer {api_key}", "Content-Type":"application/json"},
  json={
    "model": "deepseek-chat",
    "messages": [{"role":"user","content": prompt}],
    "temperature": 0.2
  },
  timeout=120
)
print("Status:", r.status_code)
r.raise_for_status()

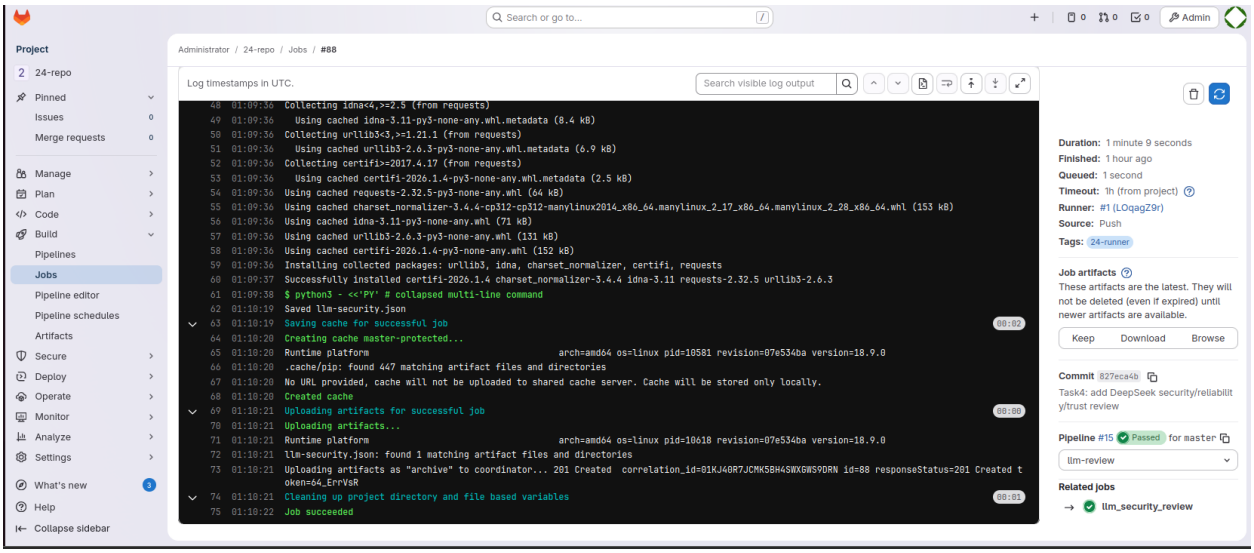
open("llm-output.json","w",encoding="utf-8").write(json.dumps(r.json(), indent=2))
print("Saved llm-output.json")
PY
  artifacts:
    when: always
    paths:
      - llm-output.json
    expire_in: 14 days
  tags: ["24-runner"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "master"'
```

Then, using an inline Python script, I read backend/main.py with the API key from the *DEEPSEEK_API_KEY* environment variable and create a prompt where LLM explains in simple terms what the application does and suggests some improvements. This prompt is sent to *DeepSeek*'s OpenAI-compatible endpoint *https://api.deepseek.com/chat/completions* and if the response is successful, it is written to a file called *llm-output.json*. I added *llm-output.json* to the artifacts section (when: always) so that the output is saved regardless of whether the job passes or fails and can be shown in the report. The job is run with a specific runner tag (*24-runner*) and rules are set to run only on the master branch. After making the changes, I *add .gitlab-ci.yml* from VM2, commit and push, resulting in a new pipeline trigger and the *llm_review* job runs successfully there.

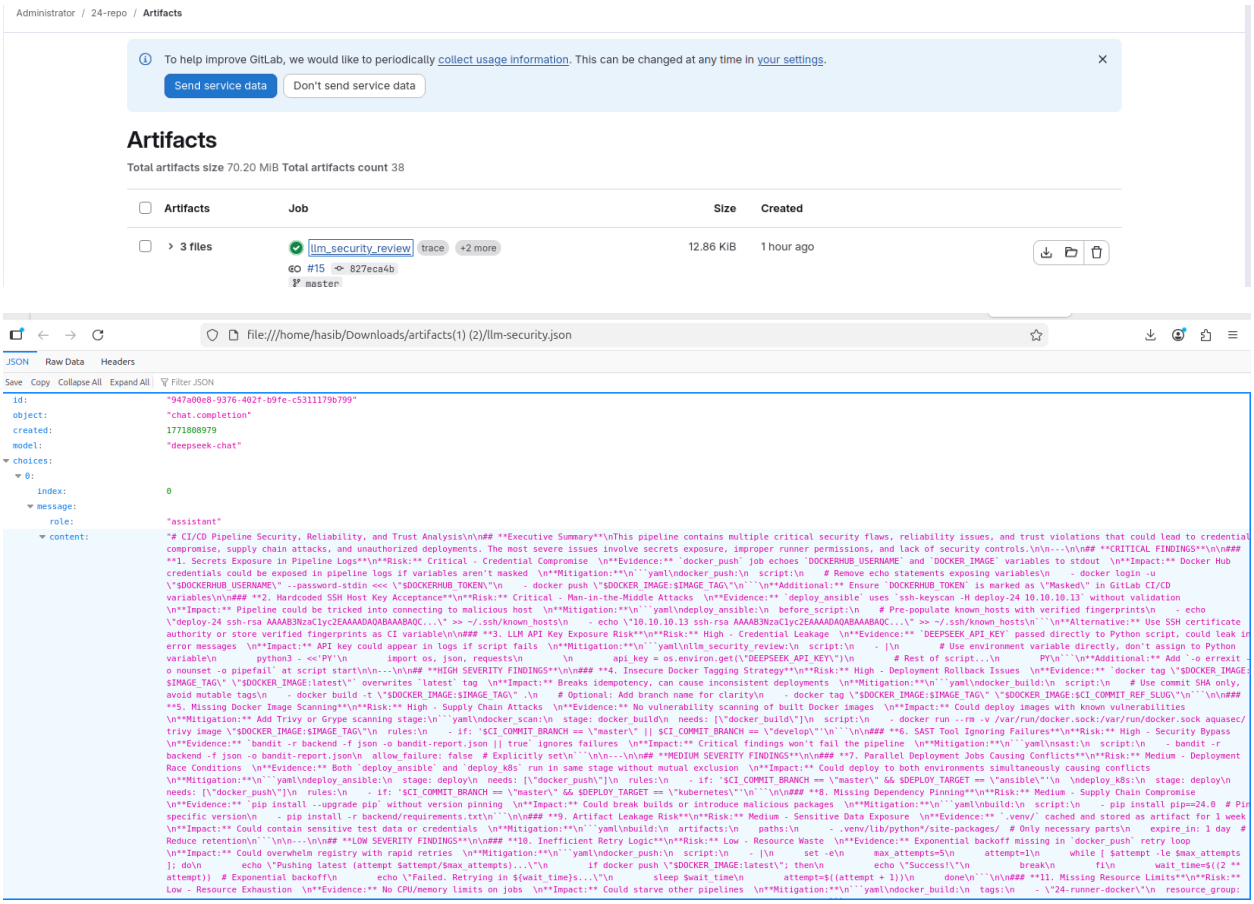
```
(.venv_ds) hasib@runner-24:~/24-repo$ git commit -m "Task4: add DeepSeek security/reliability/trust review"
[master 827eca4] Task4: add DeepSeek security/reliability/trust review
 1 file changed, 23 insertions(+), 23 deletions(-)
```

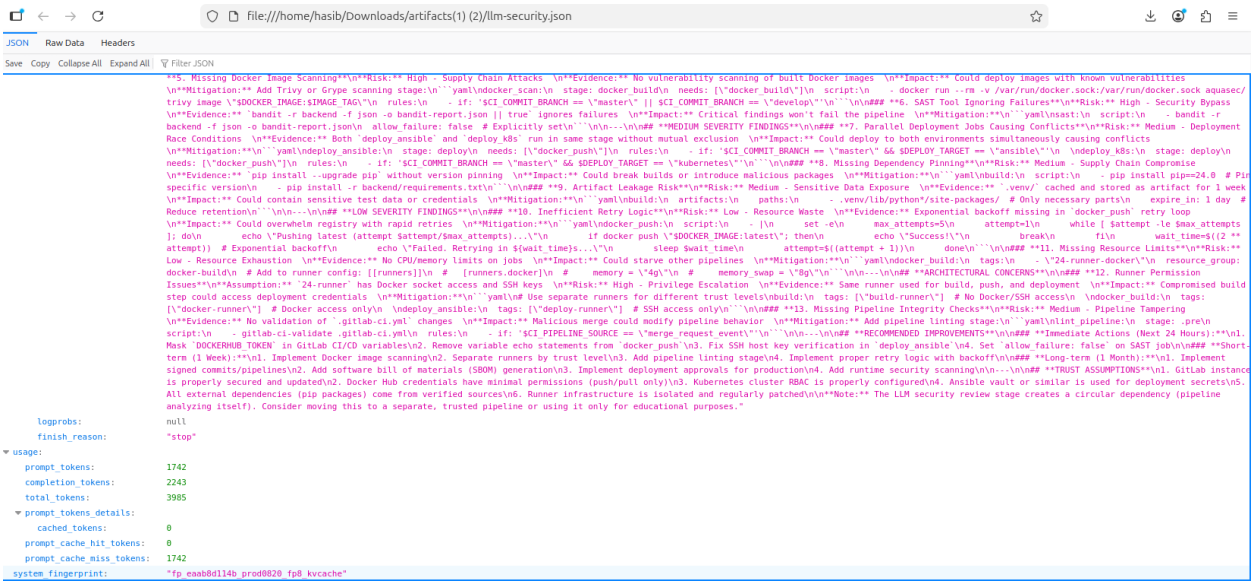
```
(.venv_ds) hasib@runner-24:~/24-repo$ git push
Username for 'http://gitlab-24.sne.com:8090': root
Password for 'http://root@gitlab-24.sne.com:8090':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 921 bytes | 115.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To http://gitlab-24.sne.com:8090/root/24-repo.git
   f5b7a9c..827eca4  master -> master
```

From VM1 (GitLab UI), I go to Pipelines > Latest pipeline to see that the *llm-review* stage is present in the updated stage list, and I open the *llm_review* job and take a screenshot showing *Status: 200* and Saved *llm-output.json* in the log.



Also, I downloaded *llm-output.json* from the Artifacts section and save a screenshot of the response output.





This way I have proven that the new *llm-review* stage has been added to the pipeline, the source/config has been sent to LLM, and the LLM response has been successfully saved as an artifact.

Task 2.3

In my pipeline, I consciously put the *llm-review* stage after the build and test stages. The main reason for this is that if the build or test fails, it means that the code is still unstable or buggy. In this case, the analysis or suggestions that are received when sending code to LLM can be misleading, because LLM is analyzing code that is not actually viable or production-ready. Therefore, it is necessary to ensure that the build and test pass successfully first, as these act as the main quality gates.

In addition, LLM is an external dependency (external API service), which requires additional time and network calls. If the initial quality checks (*build/test*) fail, then calling LLM unnecessarily is pointless and a waste of resources. Therefore, according to the logical flow of the pipeline, core functionality and correctness should be verified first, and then higher-level analysis or improvement suggestions should be taken with LLM.

When the *build* and *test* pass, the output of LLM is much more actionable and trustworthy, because then the analysis is being performed on a viable and validated codebase. As a result, placing the LLM stage after build and test is a logical, effective, and reliable decision from a pipeline design perspective.

Task 3: Testing and Validation

1- Trigger the pipeline and verify that the LLM stage runs successfully. Provide evidence from the pipeline logs and show the result of the artifact.

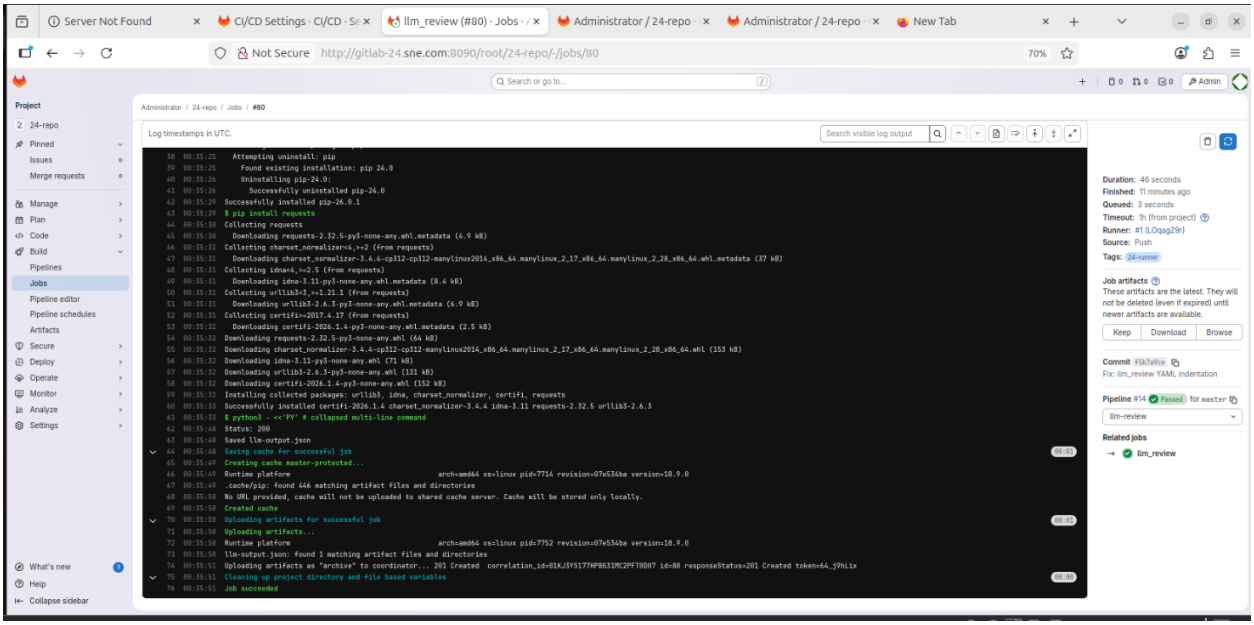
2- Does the LLM output correctly describe or explain the application or the pipeline? Briefly explain your observation.

3- How does the LLM output help a developer during CI/CD execution?

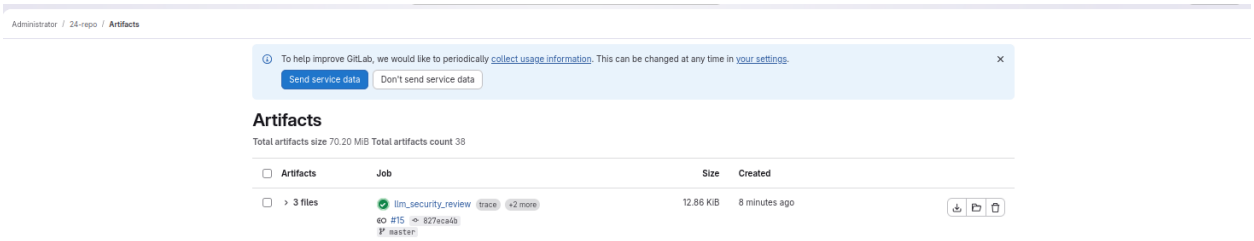
Solution:

Task 3.1

In this task, I trigger the pipeline (by issuing a new commit/push) and then open the latest pipeline from the GitLab UI on VM1 by going to *Project > Build/CI/CD > Pipelines*. From the pipeline's stage list, I click on the *llm_review* job to confirm that the LLM stage has executed successfully. The job log clearly shows *Status: 200* and *the Saved llm-output.json* message, which proves that the DeepSeek API call was successful and the LLM response file has been saved. I have included a screenshot of this part of the job log as evidence.

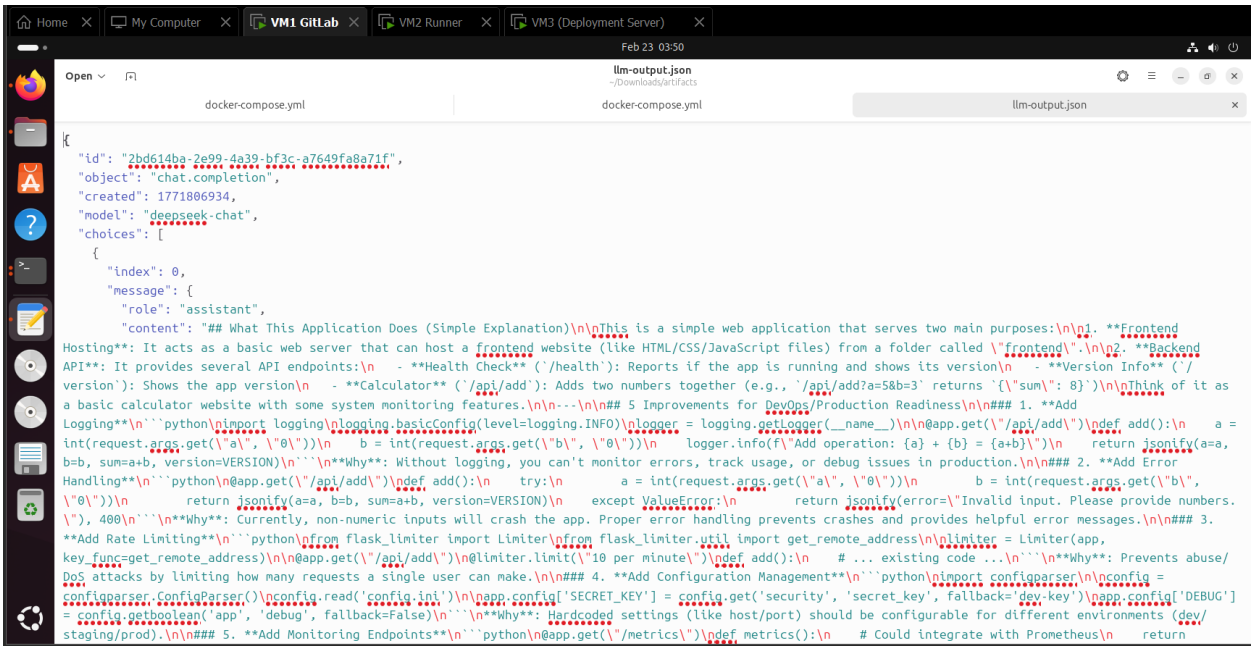


Then I go to the Artifacts section of the same job, download/open the *llm-output.json* file and see the full output of LLM there.



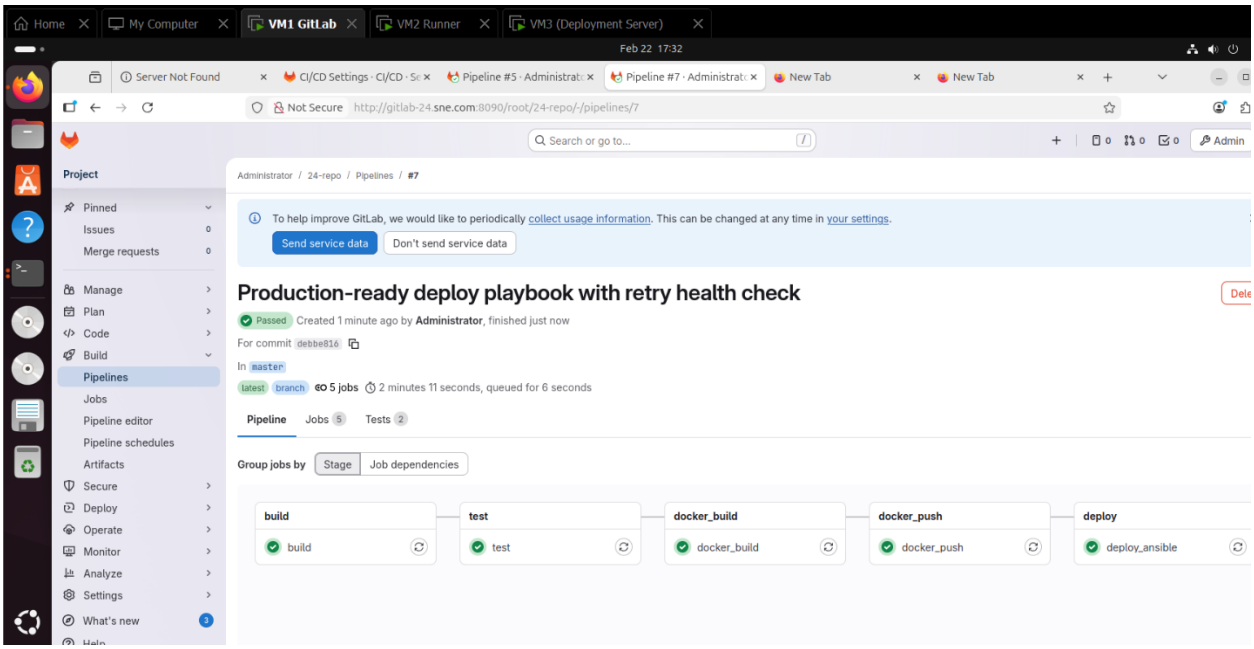
The Output contains a simple explanation of the application in the *model: "deepseek-chat" and choices[0].message.content* sections, along with 5 suggestions for improving DevOps/production readiness. For example, LLM says the application is a simple web app that serves a frontend and exposes some API endpoints (such as */health, /version, and /api/add*), and recommends adding logging, error handling, rate limiting, configuration management, and monitoring endpoints for improvement. As evidence, I have saved a screenshot of the *llm-output.json* downloaded/opened from Artifacts.





This way I have proven that the *llm-review* stage ran successfully after the pipeline was triggered and was correctly saved in the pipeline as an LLM response artifact.

Pipeline:



Task 3.2

In my observation, the explanation given by DeepSeek in *llm-output.json* describes the application fairly accurately. The output says that it is a simple web app that can serve static content from a frontend folder and provides some endpoints on the backend, specifically */health, /version*, and */api/add* (adding two numbers to get the result) - these match the behavior of my code. So, the application explanation part is correct and understandable, especially the summary given as "*basic health/version endpoint*" is appropriate.

In addition, the 5 improvements that LLM provided - adding logging, adding error handling, rate limiting, configuration management, and monitoring/metrics endpoint - are also practical and usable in terms of CI/CD and production readiness. Implementing these improves troubleshooting, monitoring, and security hardening after deploying the app.

However, one limitation is that this output is mainly application code-based; it does not talk much about the stages/CI config of the pipeline (because the prompt was mainly code). Also, the "*frontend hosting*" part is in the frontend folder/config in my repo. The overall explanation is mostly correct, but a few points need to be verified based on the codebase structure.

Task 3.3

During CI/CD execution, LLM's output acts as a quick and effective support tool for developers. It automatically provides a simple explanation of the code, so that a new team member or reviewer can quickly understand what the application does and what its main endpoints or functions are. Instead of manual code reading, LLM provides a short summary, which saves time.

LLM usually provides improvement suggestions, such as adding logging, error handling, rate limiting, or configuration management. These help developers increase production readiness and maintainability. When this feedback is automatically available in the CI/CD pipeline, developers can see potential areas for improvement as soon as they commit. LLM's analysis can often provide CI/CD troubleshooting hints. For example, if the configuration is hardcoded or error handling is missing, there may be problems after deployment - LLM can identify these issues in advance and warn. As a result, the developer is aware of potential runtime problems even after the pipeline passes.

LLM may initially provide some security hints, such as the need to add input validation, rate limiting, or monitoring endpoints. Although it is not a complete security scanner, it still makes developers aware of security risks and encourages them to follow secure coding practices.

Task 4: Security and Trust Analysis

1- what security risks can arise when sending source code, logs or configurations to an LLM?

2a- Modify your current LLM prompt so that the model analyzes the CI/CD pipeline from a security, reliability, and trust perspective. The prompt should explicitly ask the LLM to identify potential risks, vulnerabilities, and failure points, and to justify its observations.

b- Run the refined prompt, report the LLM's output, and critically assess which points you consider valid, questionable, or incorrect

Solution:

Task 4.1

By adding the *llm-review* stage to my pipeline, the application source code and some configuration parts are sent to an external LLM API. While this process is effective, it involves some significant security and trust risks that need to be considered.

The biggest risk is *Source Code* or *Intellectual Property* (*IP*) leakage. When all or part of the source code is sent to an external API, it is processed on a third-party infrastructure. If the vendor's data handling policy is not transparent or secure enough, there is a possibility of proprietary logic, business rules or internal architecture being leaked.

Secrets exposure is a major risk. Sometimes source code, logs or configuration files may contain API keys, database credentials, tokens or internal URLs. If these are sent directly to LLM without sanitizing, then sensitive information can be leaked to the external service. Although we have kept the key masked/hidden in the CI/CD variable, accidental inclusion can still be risky.

There are compliance and PII (*Personally Identifiable Information*) risks. Logs containing user data, email, IP address or other personal information and sending them to LLM could potentially violate *GDPR* or other data protection regulations. This risk is particularly high when sending production logs.

Prompt injection risk may arise. If the code or log contains malicious text (such as a string that gives the LLM the wrong instructions), the LLM may be confused and give incorrect analysis or unexpected output. This is a new type of AI-specific security risk.

Hallucination risk in LLM may sometimes give incorrect or partially incorrect advice (such as suggesting a fix that is not related to the actual problem). If the developer blindly trusts and implements that advice, new bugs or security vulnerabilities may be created.

Vendor data retention and policy risk are considerations. If it is not clear how long the external LLM provider stores data, whether it is used for training, or whether it is shared with third-parties, these issues can create long-term trust issues.

Although the LLM stage helps developers, there are risks associated with sending source code, logs, and configuration to external LLMs, including data leakage, secret exposure, compliance violation, hallucination, and vendor trust. Therefore, it is important to ensure data sanitization, minimal data sharing, and vendor policy verification when using this stage.

Task 4.2a

In this step, I changed the prompt of my existing *llm-review* stage so that LLM does not just explain the application, but also directly analyzes the CI/CD pipeline configuration from a Security, Reliability, and Trust perspective. To do this, I updated the *.gitlab-ci.yml* file in VM2 and added a new job called *llm_security_review (stage: llm-review),* where the contents of the *.gitlab-ci.yml* file are read and sent to the DeepSeek API during the pipeline. Inside the job, I create the virtual environment with *python3 -m venv .venv_llmsec* as before to avoid the PEP668 issue, activate it, and install requests. Then, using an inline Python script, I read the *.gitlab-ci.yml* file, get the API key from the *DEEPSEEK_API_KEY* environment variable, and create a refined prompt. In this prompt, I have explicitly instructed the model to analyze the pipeline from a *SECURITY, RELIABILITY, TRUST* perspective and (1) identify potential risk, vulnerability, failure points, (2) provide a brief justification for each point, (3) propose concrete mitigation with YAML, (4) flag secrets handling and supply-chain issues separately, and (5) indicate what is missing if the information is insufficient.



I then send this prompt to DeepSeek's OpenAI-compatible endpoint *https://api.deepseek.com/chat/completions* and if the response is successful, save it as *llm-security.json* so that it is available as a pipeline artifact. I added *llm-security.json* to the artifacts section (*when: always*) and set the retention to *14 days*. The job is run using the 24-runner tag and the rules are set to run only on the master branch.

```
      PIPELINE:
      {pipeline[:14000]}
      """

      r = requests.post(
        "https://api.deepseek.com/chat/completions",
        headers={"Authorization": f"Bearer {api_key}", "Content-Type":"application/json"},
        json={"model":"deepseek-chat","messages":[{"role":"user","content":prompt}],"temperature":0.2},
        timeout=180
      )

      r.raise_for_status()
      open("llm-security.json","w",encoding="utf-8").write(json.dumps(r.json(), indent=2))
      print("Saved llm-security.json")
      PY
  artifacts:
    when: always
    paths:
      - llm-security.json
    expire_in: 14 days
  tags: ["24-runner"]
  rules:
    - if: '$CI_COMMIT_BRANCH == "master"'


^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

Then, I add the *.gitlab-ci.yml* file, commit and push, which triggers the new pipeline and prepares the LLM security review job to run with refined prompts.

add:

```
(.venv_ds) hasib@runner-24:~/24-repo$ git add .gitlab-ci.yml
```

commit:

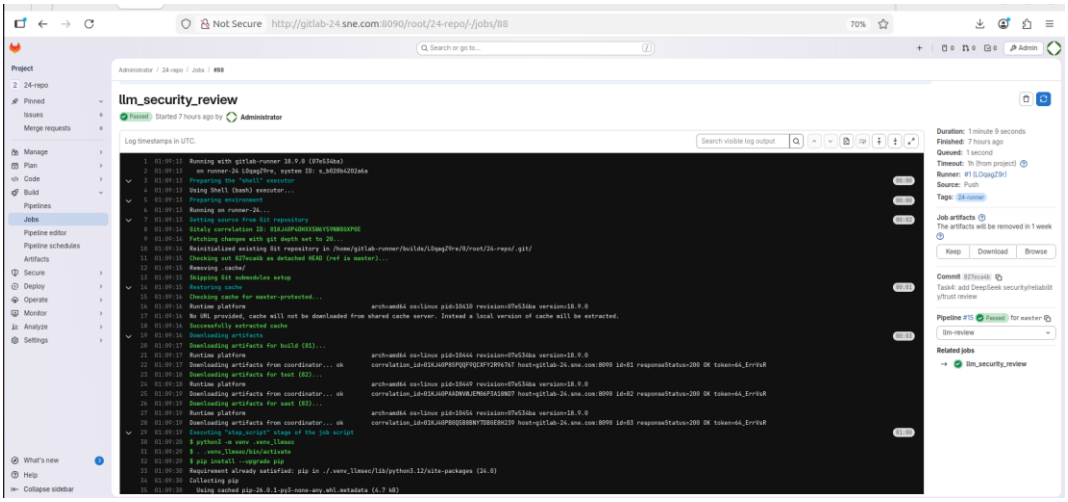
```
(.venv_ds) hasib@runner-24:~/24-repo$ git commit -m "Task4: add DeepSeek security/reliability/trust review"
[master 827eca4] Task4: add DeepSeek security/reliability/trust review
 1 file changed, 23 insertions(+), 23 deletions(-)
```

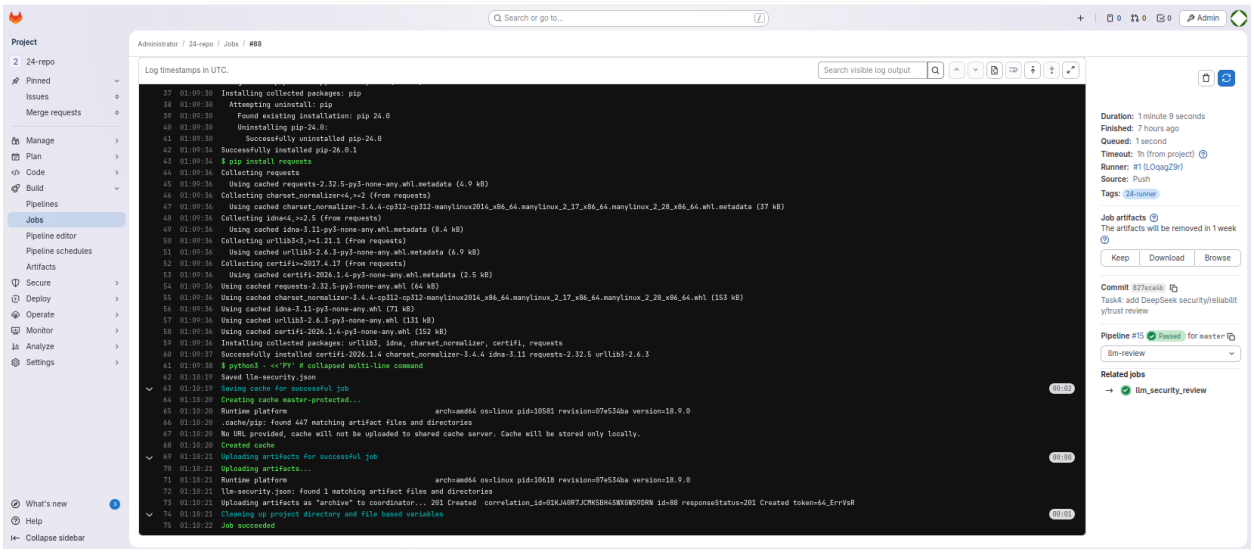
push:

```
(.venv_ds) hasib@runner-24:~/24-repo$ git push
Username for 'http://gitlab-24.sne.com:8090': root
Password for 'http://root@gitlab-24.sne.com:8090':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 921 bytes | 115.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
To http://gitlab-24.sne.com:8090/root/24-repo.git
   f5b7a9c..827eca4  master -> master
```

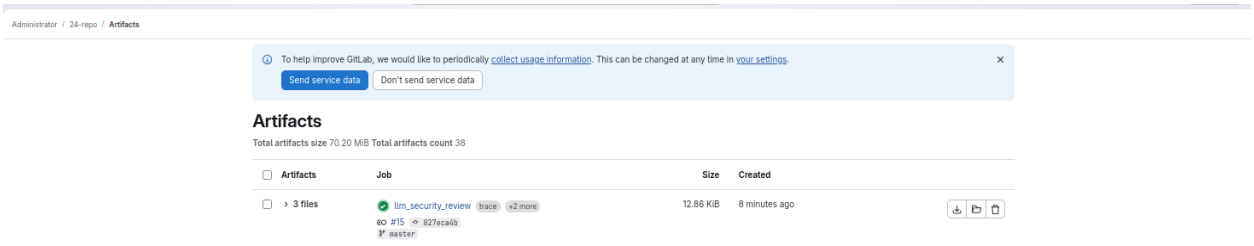
Task 4.2b

After adding the refined prompt, I trigger the pipeline and open the latest pipeline by going to Project from the GitLab UI of VM1, then from CI/CD to Pipelines. There, the *llm_security_review* job has run successfully and the job log shows Status: 200, job succeeded and Saved *llm-security_review.json.*

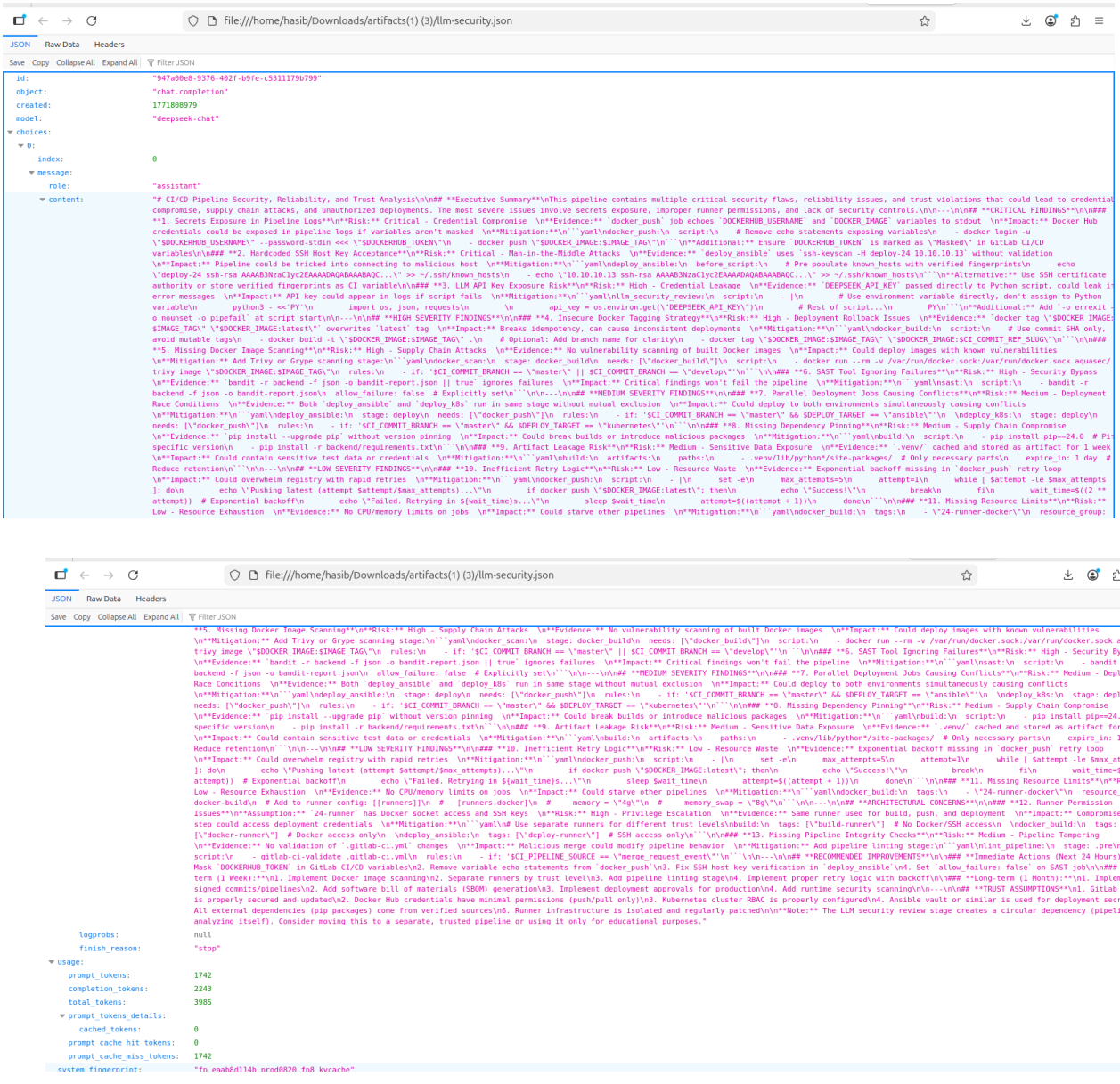




Then, download *llm-security.json* from the Artifacts section and review the security, reliability, and trust analysis output of LLM.



As evidence, I have saved a screenshot of the job log and artifact download.



The LLM output identifies various security, reliability, and trust-related risks to the pipeline and provides mitigation recommendations, including YAML, for each. Below, I present a critical assessment of them.

Valid Observations

LLM correctly identified the secrets handling risk, especially the *DockerHub* credential and *DEEPSEEK_API_KEY* exposure potential. Secret masking and least privilege are very important in a CI/CD environment, so this observation is appropriate.

LLM stated that the SAST failure pipeline is not failing because of *bandit ... || true* . This is absolutely correct, as it allows deployment to proceed even if it has a security vulnerability.

The lack of Docker image vulnerability scanning is mentioned as a valid supply-chain risk. The suggestion to add *Trivy/Grype* is realistic.

The use of mutable latest tag can cause rollback and reproducibility issues, this analysis is also correct according to DevOps best practice.

It makes sense from an architectural perspective to identify the privilege escalation risk of building and deploying with the same runner.

While advice like dependency pinning and pipeline linting are best practices, they do not directly prove a critical vulnerability in my pipeline - they are somewhat generic *DevSecOps* advice.

Incorrect

LLM made assumptions in some places that weren't in the pipeline. For example, it claimed *.venv/ artifact* leakage, which wasn't in my configuration.

It also made assumptions about parallel deployment conflicts, even though deployment was controlled by rules. Retry logic and some structural concerns were generic suggestions, not directly evidence-based.

Bonus Task: Advanced Use of Large Language Models in DevOps

1- Deploy a lightweight LLM model (phi-3, llama3, mistral-quantized). Show the commands used to install the runtime and start the model.

Hint: ***Hugging Face, for AI you can use llama3***

2- Extend the CI/CD pipeline so the LLM performs advanced analysis. The LLm should be able to:

- **Analyze logs from a failed CI/CD job and suggest possible fixes.**
- **Detects outdated dependencies or code quality issues.**
- **Provide concise improvement suggestions for documentation on the README file.**

3- Design a workflow where the LLM produces a summary of detected issues or improvements and creates a pull (merge) request.

Solution:

Bonus 1

In Bonus 1, I deployed a lightweight local LLM model, which allows for local inference without external APIs. I did this on VM2 (*runner-24*) due to the high resource requirements, although it may be a bit slow due to CPU-only mode. I used DeepSeek in the main pipeline, but this is an optional demo to show local LLM deployment.

First, I *installed* the *Ollama runtime* using the following command:

```
(.venv_ds) hasib@runner-24:~/24-repo$ curl -fsSL https://ollama.com/install.sh | sh
>>> Cleaning up old version at /usr/local/lib/ollama
[sudo] password for hasib:
Sorry, try again.
[sudo] password for hasib:
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
######################################################################## 100.0%
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.
```

The terminal output shows that *Ollama* has been successfully installed in */usr/local,* the *systemd* service has been enabled, and the API endpoint is available at *127.0.0.1:11434*. It also shows a

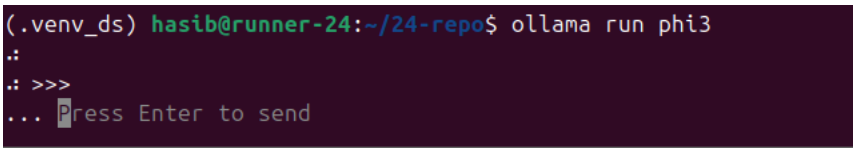
warning that no NVIDIA/AMD GPU is found, so it will run in CPU-only mode, which is normal for my VM environment.

Then I *pull phi-3* as a lightweight model:



The output shows a download and success message, confirming that the model has been installed on the local machine.

Then run the model and verify:



The output shows a download and success message, confirming that the model has been installed on the local machine.

Bonus 2

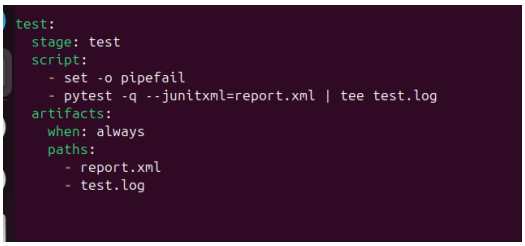
In this task, I extended my CI/CD pipeline in such a way that LLM can perform three types of advanced analysis -

(1) analyze the log of a job failure to suggest root causes and fixes,

(2) detect outdated dependencies or code quality issues, and

(3) provide concise improvement suggestions for *README* documentation.

To implement the first part, I set up a job output log in the test stage. For example, when running *pytest* in a test job, I use tee test.log, so that the test output is simultaneously displayed in the terminal and saved in the test.log file; and I also create a report.xml JUnit report.



Then, I set *artifacts: when: always* to *report.xml* and *test.log* as artifacts, so that the log file is available in both cases, whether the pipeline passes or fails.

```
 GNU nano 7.2                                                    .gitlab-ci.yml *
llm_debug_on_failure:
  stage: llm-review
  when: on_failure
  script:
    - python3 -m venv .venv_dbg
    - . .venv_dbg/bin/activate
    - pip install --upgrade pip
    - pip install requests
    - |
        python3 - <<'PY'
        import os, json, requests, glob

        api_key = os.getenv("DEEPSEEK_API_KEY")
        if not api_key:
            raise SystemExit("DEEPSEEK_API_KEY missing")

        logs = ""
        for f in sorted(glob.glob("*.log")):
            logs += f"\n\n=== {f} ===\n" + open(f,"r",encoding="utf-8",errors="ignore").read()[:12000]

        prompt = f"""
        You are a senior DevOps engineer.
        Analyze these CI job logs from a failed pipeline.
```

```
 GNU nano 7.2                                                    .gitlab-ci.yml *
        You are a senior DevOps engineer.
        Analyze these CI job logs from a failed pipeline.

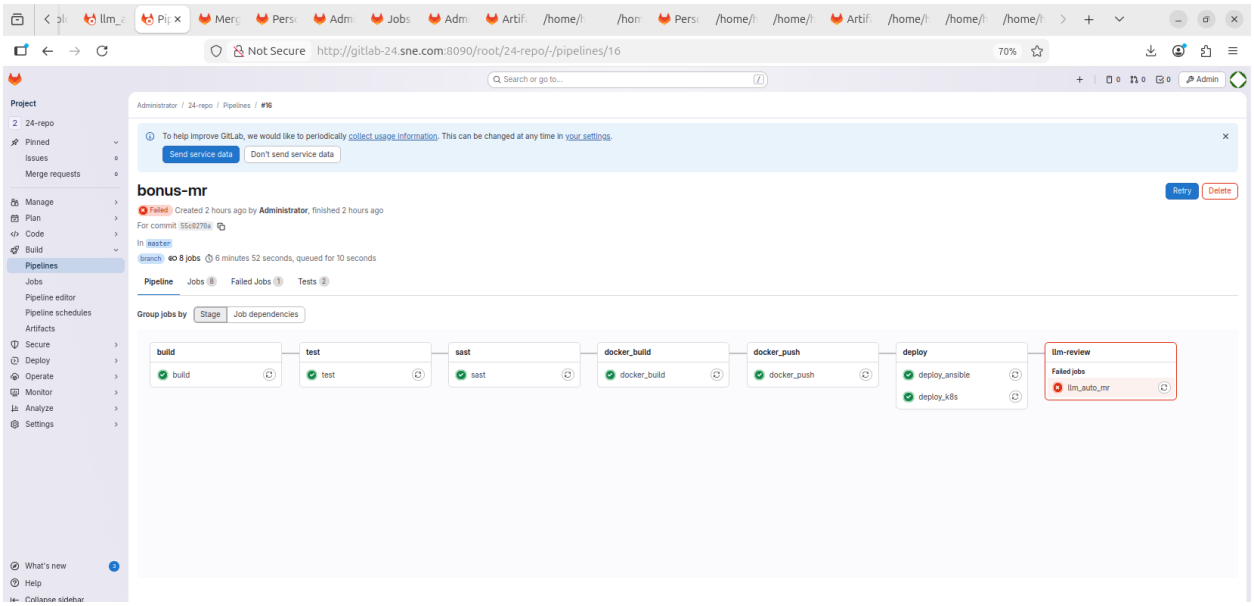
        Return:
        1) Most likely root cause
        2) 3 concrete fixes (commands or YAML changes)
        3) Evidence from logs

        LOGS:
        {logs[:20000]}
        """

        r = requests.post(
          "https://api.deepseek.com/chat/completions",
          headers={"Authorization": f"Bearer {api_key}", "Content-Type":"application/json"},
          json={"model":"deepseek-chat","messages":[{"role":"user","content":prompt}],"temperature":0.2},
          timeout=180
        )
        r.raise_for_status()
        open("llm-debug.json","w",encoding="utf-8").write(json.dumps(r.json(), indent=2))
        print("Saved llm-debug.json")
        PY
  artifacts:
    when: always
    paths:
      - llm-debug.json
  tags: ["24-runner"]

^G Help        ^O Write Out   ^W Where Is   ^K Cut         ^T Execute     ^C Location    M-U Undo
^X Exit        ^R Read File   ^\ Replace    ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo
```

Next, I add a new job called *llm_debug_on_failure,* with *when: on_failure set,* meaning that this LLM job will automatically run when a job in the pipeline fails. In this job, I create a *venv* and install requests, then read all the *.log files in the repo and send them to the DeepSeek API. In the prompt, I explicitly instruct the model to look at the failed log and (1) suggest the most likely root cause, (2) provide 3 concrete fixes (YAML changes), and (3) provide evidence from the log. The output is saved as an artifact called *llm-debug.json*, so the developer gets an AI-generated troubleshooting report as soon as the pipeline fails.




```diff
--- README.md
+++ README.md
@@ -1,56 +1,56 @@
-# 24-repo
+# Project: 24-repo

+## Overview
+This repository contains the source code and configuration for the 24-repo project. It is managed with Git and uses GitLab for CI/CD, collaboration, and project management.

-## Getting started
+## Quick Start

-To make it easy for you to get started with GitLab, here's a list of recommended next steps.
+### Prerequisites
+*   Git
+*   Access to the GitLab instance at `http://24.sne.com`

-Already a pro? Just edit this README.md and make it your own. Want to make it easy? [Use the template at the bottom](#editing-this-readme)!
+### Clone and Initialize the Repository
+```bash
+# Clone the repository
+git clone http://24.sne.com/root/24-repo.git
+cd 24-repo

-## Integrate with your tools
+### Standard Git Commands
+```bash
+# Create and switch to a new feature branch
+git checkout -b feature/your-feature-name develop

-* [Set up project integrations](http://24.sne.com/root/24-repo/-/settings/integrations)
+# Stage, commit, and push changes
+git add .
+git commit -m "Descriptive commit message"
+git push origin feature/your-feature-name
+```
+After pushing, create a Merge Request (MR) from your feature branch into `develop` via the GitLab web interface.

-## Collaborate with your team
+## CI/CD Pipeline

-* [Invite team members and collaborators](https://docs.gitlab.com/ee/user/project/members/)
-* [Create a new merge request](https://docs.gitlab.com/ee/user/project/merge_requests/creating_merge_requests.html)
-* [Automatically close issues from merge requests](https://docs.gitlab.com/ee/user/project/issues/managing_issues.html#closing-issues-automatically)
-* [Enable merge request approvals](https://docs.gitlab.com/ee/user/project/merge_requests/approvals/)
-* [Set auto-merge](https://docs.gitlab.com/user/project/merge_requests/auto_merge/)
+The project uses GitLab CI/CD. The pipeline configuration is defined in `.gitlab-ci.yml` at the root of the repository.
```

```
-* [Use pull-based deployments for improved Kubernetes management](https://docs.gitlab.com/ee/user/clusters/agent/)
-* [Set up protected environments](https://docs.gitlab.com/ee/ci/environments/protected_environments.html)
-
-***
-
-# Editing this README
-
-When you're ready to make this README your own, just edit this file and use the handy template below (or feel free to structure it however you want - this is
just a starting point!). Thanks to [makeareadme.com](https://www.makeareadme.com/) for this template.
-
-## Suggestions for a good README
-
-Every project is different, so consider which of these sections apply to yours. The sections used in the template are suggestions for most open source projects.
Also keep in mind that while a README can be too long and detailed, too long is better than too short. If you think your README is too long, consider utilizing
another form of documentation rather than cutting out information.
-
-## Name
-Choose a self-explaining name for your project.
-
-## Description
-Let people know what your project can do specifically. Provide context and add a link to any reference visitors might be unfamiliar with. A list of Features or a
Background subsection can also be added here. If there are alternatives to your project, this is a good place to list differentiating factors.
-
-## Badges
-On some READMEs, you may see small images that convey metadata, such as whether or not all the tests are passing for the project. You can use Shields to add some
to your README. Many services also have instructions for adding a badge.
-
-
-## Contributing
-State if you are open to contributions and what your requirements are for accepting them.
-
-For people who want to make changes to your project, it's helpful to have some documentation on how to get started. Perhaps there is a script that they should
run or some environment variables that they need to set. Make these steps explicit. These instructions could also be useful to your future self.
-
-You can also document commands to lint the code or run tests. These steps help to ensure high code quality and reduce the likelihood that the changes
inadvertently break something. Having instructions for running tests is especially helpful if it requires external setup, such as starting a Selenium server for
testing in a browser.
-
-## Authors and acknowledgment
-Show your appreciation to those who have contributed to the project.
-
-## License
-For open source projects, say how it is licensed.
-
-## Project status
-If you have run out of energy or time for your project, put a note at the top of the README saying that development has slowed down or stopped completely.
Someone may choose to fork your project or volunteer to step in as a maintainer or owner, allowing your project to keep going. You can also make an explicit
request for maintainers.
-change on develop
+*   **Issues:** Track tasks, enhancements, and bugs under **Issues** in GitLab.
+*   **Merge Requests (MRs):** Review and merge code changes. Link MRs to issues.
+*   **Wiki:** Project documentation can be maintained in the GitLab Wiki.
+*   **Settings:** Configure integrations, webhooks, and access permissions under **Settings**.
+*   **Members:** Manage team access under **Project Information > Members**.
```

In the second part, I add a job called *deps_scan* to detect outdated dependencies or code quality issues. Here, I create a *venv*, install the requirements, and run pip list --*outdated* > *outdated.txt* || *true* to extract the outdated package list. I also save this outdated.txt file as an artifact, so that at the end of the pipeline run, the developer can easily understand which dependency upgrades are needed. If desired, a vulnerability audit report can also be artifact by adding pip-audit to the same stage. This provides additional benefits for supply-chain security.

```
deps_scan:
  stage: test
  script:
    - python3 -m venv .venv_deps
    - . .venv_deps/bin/activate
    - pip install --upgrade pip
    - pip install -r backend/requirements.txt
    - pip list --outdated > outdated.txt || true
  artifacts:
    when: always
    paths:
      - outdated.txt
  tags: ["24-runner"]
```

```
llm_readme_suggest:
  stage: llm-review
  script:
    - python3 -m venv .venv_readme
    - . .venv_readme/bin/activate
    - pip install --upgrade pip
    - pip install requests
    - |
        python3 - <<'PY'
        import os, json, requests

        api_key = os.getenv("DEEPSEEK_API_KEY")
        if not api_key:
            raise SystemExit("DEEPSEEK_API_KEY missing")

        readme = open("README.md","r",encoding="utf-8",errors="ignore").read()

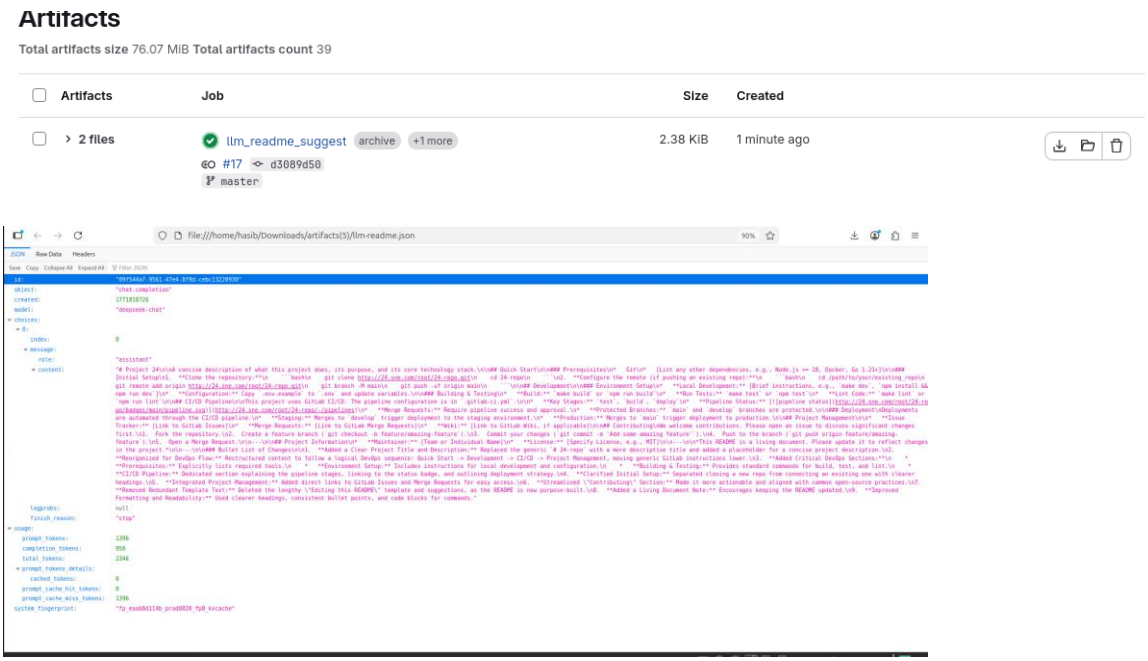
        prompt = f"""
        Improve this README for clarity and DevOps friendliness.
        Return:
        1) Improved README text
        2) Bullet list of changes

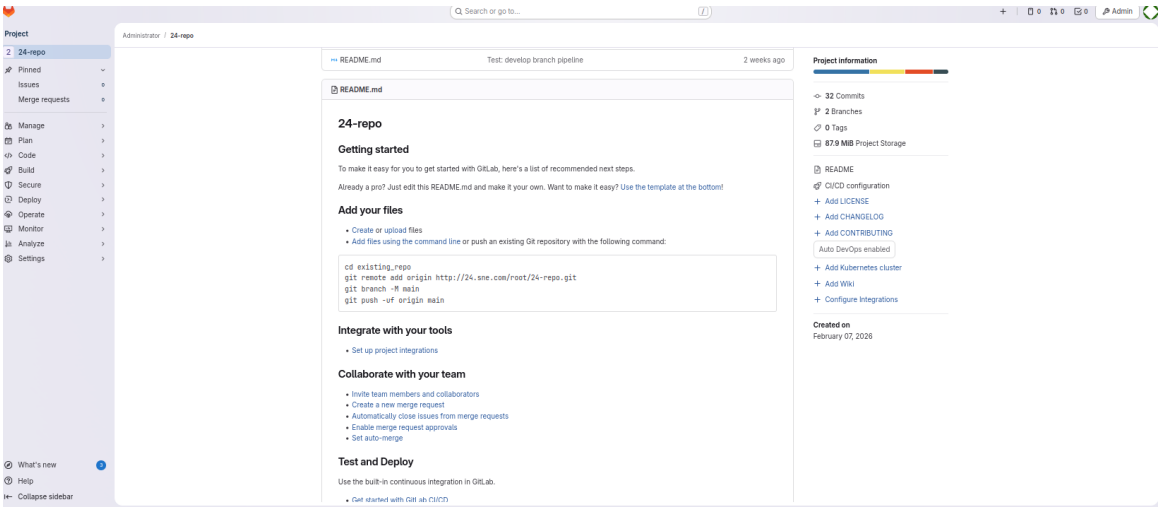
        README:
        {readme[:14000]}
        """
```

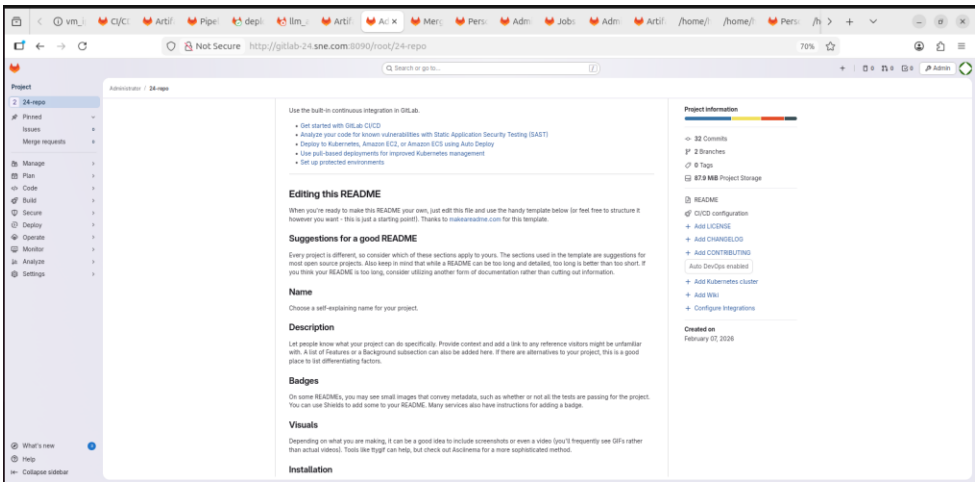
```
        r = requests.post(
          "https://api.deepseek.com/chat/completions",
          headers={"Authorization": f"Bearer {api_key}", "Content-Type":"application/json"},
          json={"model":"deepseek-chat","messages":[{"role":"user","content":prompt}],"temperature":0.2},
          timeout=180
        )
        r.raise_for_status()
        open("llm-readme.json","w",encoding="utf-8").write(json.dumps(r.json(), indent=2))
        print("Saved llm-readme.json")
        PY
  artifacts:
    when: always
    paths:
      - llm-readme.json
  tags: ["24-runner"]
```

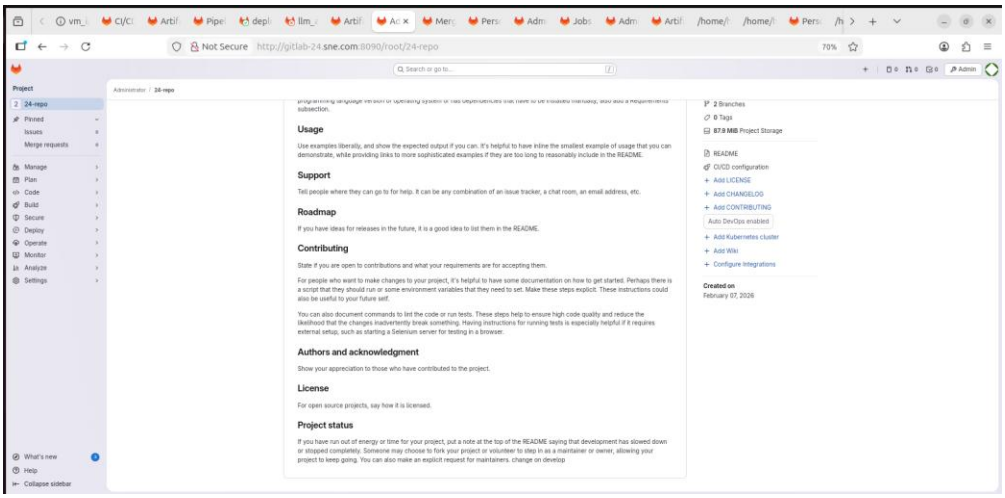
In the third part, I added the *llm_readme_suggest* job to improve the *README* documentation, which uses *venv* + *requests* to read the *README.md* file and send it to DeepSeek. The prompt asks the model to rewrite the *README* to be DevOps-friendly and provide a short list of the changes in bullet list format. After the pipeline runs, the improved *README* text and bullet list of the model are available in the *llm-readme.json* artifact. My artifact output shows that the model has organized the *README* according to the Project Management flow from Quick Start to Development to CI/CD, suggested adding prerequisites, *build/test/lint* commands, and cleaned up the documentation by removing unnecessary template text, which directly matches this requirement.

Evidence: *llm-readme_Suggest.json artifact*

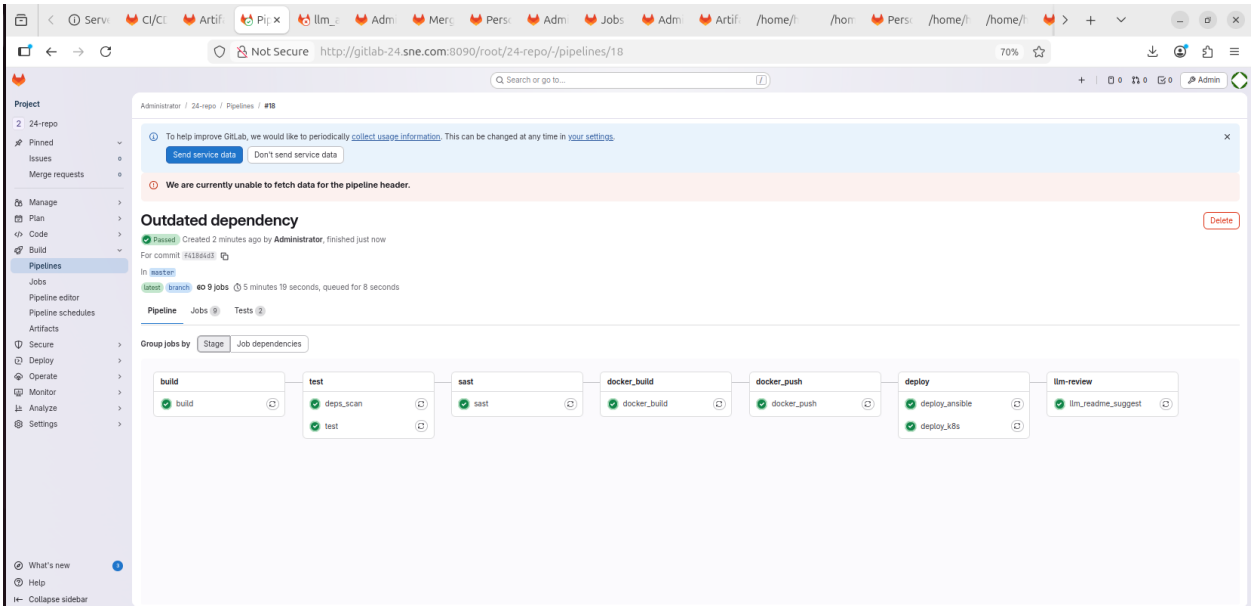






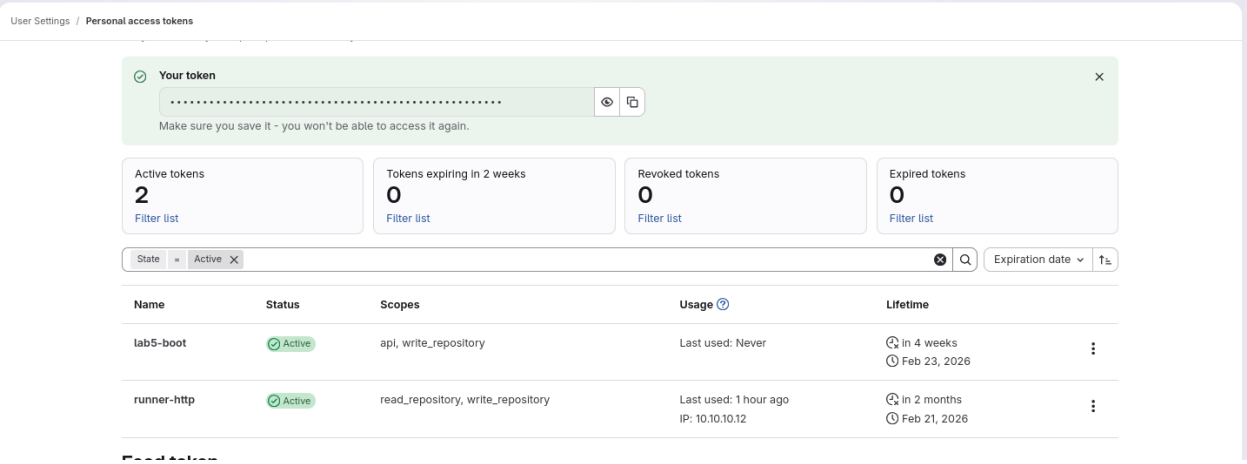


Pipeline:

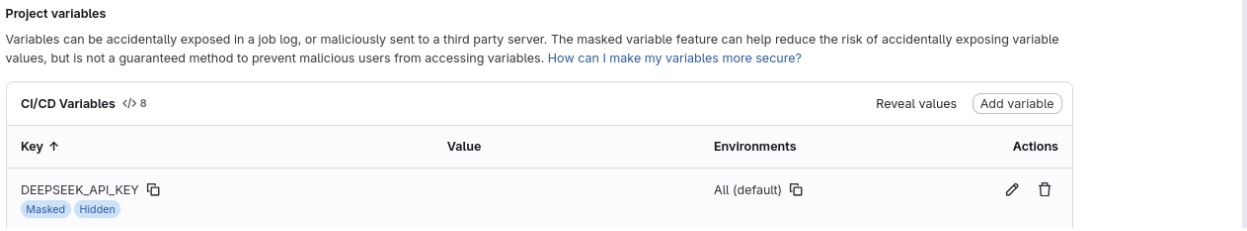


Bonus 3

In Bonus 3, I designed a workflow where LLM creates a summary of detected issues or improvements (especially *README* documentation improvements) within the pipeline, then pushes those changes to a new branch and automatically creates a Merge Request (MR). I completed this workflow in three steps. First, I created an Access Token from the GitLab profile on VM1 (Profile > Access Tokens), where I selected *api* and *write_repository* as scopes and named the token lab5-bot. After the token was created, I copied it and saved it safely, because this token is needed to push the branch from the pipeline and create the MR.

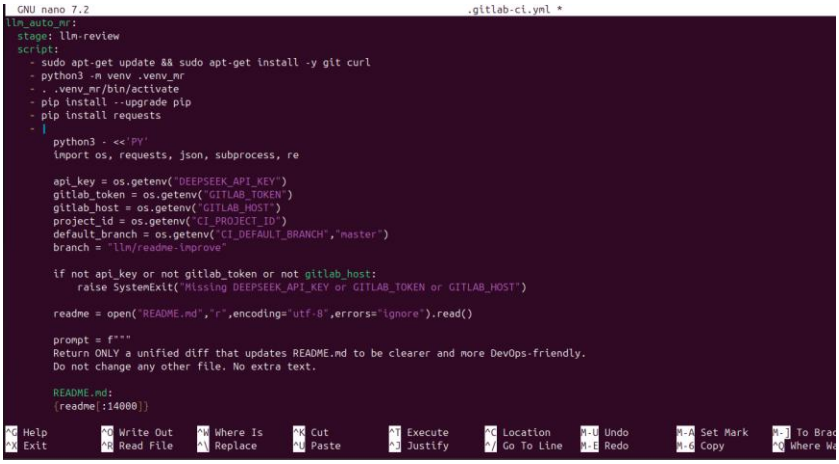


In the second step, I added the required variables from VM1's GitLab UI by going to Project, Settings, CI/CD, and then Variables. Here, I set the token value in the variable named *GITLAB_TOKEN* and enabled *Masked* and *Hidden*, so that the token does not leak in the pipeline log. I also set *http://gitlab-24.sne.com:8090* as *GITLAB_HOST*, so that the job can find the GitLab API endpoint correctly. (The DeepSeek key was previously set as *DEEPSEEK_API_KEY*.)



In the third step, I added a job called *llm_auto_mr* to the *.gitlab-ci.yml* file on VM2 (*stage: llm-review*), which will run only on the master branch. The job initially installs the necessary tools (git, curl) and creates a Python *venv* to install requests. Then the Python script inside the job first reads *README.md* and sends a prompt to the DeepSeek API, which clearly indicates that the model should only return unified diffs and only update the *README.md* file, not additional text. The diff obtained from LLM is saved in the *readme.patch* file, then a new branch (*llm/readme-improve*) is checked out with git config set, patch applied, *README* changes committed, and force pushed to that branch remote using the *OAuth* token from the pipeline. Finally, a new MR is created from the source branch to the default branch using the *GitLab REST API*

(/api/v4/projects/{project_id}/merge_requests) and its response is saved as an artifact as *mr-response.json*.



```
  GNU nano 7.2                                                  .gitlab-ci.yml *
llm_auto_mr:
  stage: llm-review
  script:
    - sudo apt-get update && sudo apt-get install -y git curl
    - python3 -m venv .venv_mr
    - . .venv_mr/bin/activate
    - pip install --upgrade pip
    - pip install requests
    - |
        python3 - <<'PY'
        import os, requests, json, subprocess, re

        api_key = os.getenv("DEEPSEEK_API_KEY")
        gitlab_token = os.getenv("GITLAB_TOKEN")
        gitlab_host = os.getenv("GITLAB_HOST")
        project_id = os.getenv("CI_PROJECT_ID")
        default_branch = os.getenv("CI_DEFAULT_BRANCH","master")
        branch = "llm/readme-improve"

        if not api_key or not gitlab_token or not gitlab_host:
            raise SystemExit("Missing DEEPSEEK_API_KEY or GITLAB_TOKEN or GITLAB_HOST")

        readme = open("README.md","r",encoding="utf-8",errors="ignore").read()

        prompt = f"""
        Return ONLY a unified diff that updates README.md to be clearer and more DevOps-friendly.
        Do not change any other file. No extra text.

        README.md:
        {readme[:14000]}

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Brac
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Wa
```



```
        README.md:
        {readme[:14000]}
        """

        ds = requests.post(
          "https://api.deepseek.com/chat/completions",
          headers={"Authorization": f"Bearer {api_key}", "Content-Type":"application/json"},
          json={"model":"deepseek-chat","messages":[{"role":"user","content":prompt}],"temperature":0.2},
          timeout=180
        ).json()

        diff_text = ds["choices"][0]["message"]["content"]
        open("readme.patch","w",encoding="utf-8").write(diff_text)

        subprocess.run(["git","config","user.email","ci-bot@local"], check=True)
        subprocess.run(["git","config","user.name","ci-bot"], check=True)

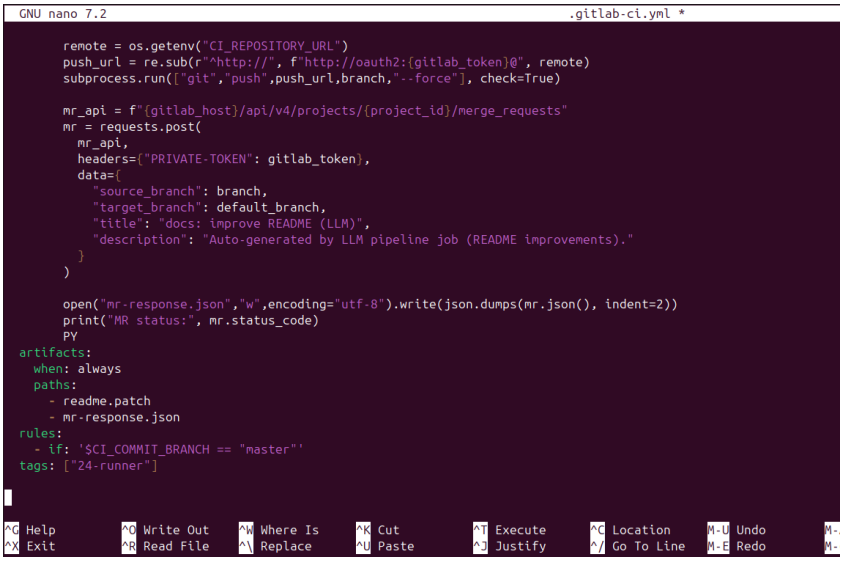
        subprocess.run(["git","checkout","-B",branch], check=True)
        subprocess.run(["git","apply","--check","readme.patch"], check=True)
        subprocess.run(["git","apply","readme.patch"], check=True)

        subprocess.run(["git","add","README.md"], check=True)
        subprocess.run(["git","commit","-m","docs: improve README (LLM)"], check=True)

        remote = os.getenv("CI_REPOSITORY_URL")
        push_url = re.sub(r"^http://", f"http://oauth2:{gitlab_token}@", remote)
        subprocess.run(["git","push",push_url,branch,"--force"], check=True)

        mr_api = f"{gitlab_host}/api/v4/projects/{project_id}/merge_requests"

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Se
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Co
```



```
  GNU nano 7.2                                                  .gitlab-ci.yml *
        remote = os.getenv("CI_REPOSITORY_URL")
        push_url = re.sub(r"^http://", f"http://oauth2:{gitlab_token}@", remote)
        subprocess.run(["git","push",push_url,branch,"--force"], check=True)

        mr_api = f"{gitlab_host}/api/v4/projects/{project_id}/merge_requests"
        mr = requests.post(
          mr_api,
          headers={"PRIVATE-TOKEN": gitlab_token},
          data={
            "source_branch": branch,
            "target_branch": default_branch,
            "title": "docs: improve README (LLM)",
            "description": "Auto-generated by LLM pipeline job (README improvements)."
          }
        )

        open("mr-response.json","w",encoding="utf-8").write(json.dumps(mr.json(), indent=2))
        print("MR status:", mr.status_code)
        PY
  artifacts:
    when: always
    paths:
      - readme.patch
      - mr-response.json
  rules:
    - if: '$CI_COMMIT_BRANCH == "master"'
  tags: ["24-runner"]

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-
```

After the pipeline ran, I went to Project on VM1 and saw the newly created MR, and saved the *mr-response.json* artifact download/open screenshot and the screenshot showing the MR in the MR list as evidence.




```diff
--- README.md
+++ README.md
@@ -1,56 +1,56 @@
-# 24-repo
+# Project: 24-repo

+## Overview
+This repository contains the source code and configuration for the 24-repo project. It is managed with Git and uses GitLab for CI/CD, collaboration, and project management.

-## Getting started
+## Quick Start

-To make it easy for you to get started with GitLab, here's a list of recommended next steps.
+### Prerequisites
+*   Git
+*   Access to the GitLab instance at `http://24.sne.com`

-Already a pro? Just edit this README.md and make it your own. Want to make it easy? [Use the template at the bottom](#editing-this-readme)!
+### Clone and Initialize the Repository
+```bash
+# Clone the repository
+git clone http://24.sne.com/root/24-repo.git
+cd 24-repo
```

```diff
-Already a pro? Just edit this README.md and make it your own. Want to make it easy? [Use the template at the bottom](#editing-this-readme)!
+### Clone and Initialize the Repository
+```bash
+# Clone the repository
+git clone http://24.sne.com/root/24-repo.git
+cd 24-repo

-## Add your files
+# If pushing an existing local repository, set the remote
+git remote add origin http://24.sne.com/root/24-repo.git
+git branch -M main
+git push -uf origin main
+```

-* [Create](https://docs.gitlab.com/ee/user/project/repository/web_editor.html#create-a-file) or [upload](https://docs.gitlab.com/ee/user/project/repository/web_editor.html#upload-a-file) files
-* [Add files using the command line](https://docs.gitlab.com/topics/git/add_files/#add-files-to-a-git-repository) or push an existing Git repository with the following command:
+## Development Workflow

-```
-cd existing_repo
-git remote add origin http://24.sne.com/root/24-repo.git
-git branch -M main
-git push -uf origin main
-```
```

```
+## CI/CD Pipeline

-* [Invite team members and collaborators](https://docs.gitlab.com/ee/user/project/members/)
-* [Create a new merge request](https://docs.gitlab.com/ee/user/project/merge_requests/creating_merge_requests.html)
-* [Automatically close issues from merge requests](https://docs.gitlab.com/ee/user/project/issues/managing_issues.html#closing-issues-automatically)
-* [Enable merge request approvals](https://docs.gitlab.com/ee/user/project/merge_requests/approvals/)
-* [Set auto-merge](https://docs.gitlab.com/user/project/merge_requests/auto_merge/)
+The project uses GitLab CI/CD. The pipeline configuration is defined in `.gitlab-ci.yml` at the root of the repository.

-## Test and Deploy
+*   **Pipeline Status:** View the latest pipeline runs under **CI/CD > Pipelines** in GitLab.
+*   **Jobs & Artifacts:** Check individual job logs and download artifacts (e.g., build outputs, test reports) from the pipeline view.
+*   **Automated Tests:** The pipeline runs tests on every push to ensure code quality.
+*   **Deployment:** Successful pipelines on the `main` branch may trigger automated deployments (configured in `.gitlab-ci.yml`).

-Use the built-in continuous integration in GitLab.
+## Project Management & Collaboration

-* [Get started with GitLab CI/CD](https://docs.gitlab.com/ee/ci/quick_start/)
-* [Analyze your code for known vulnerabilities with Static Application Security Testing (SAST)](https://docs.gitlab.com/ee/user/application_security/sast/)
-* [Deploy to Kubernetes, Amazon EC2, or Amazon ECS using Auto Deploy](https://docs.gitlab.com/ee/topics/autodevops/requirements.html)
-* [Use pull-based deployments for improved Kubernetes management](https://docs.gitlab.com/ee/user/clusters/agent/)
-* [Set up protected environments](https://docs.gitlab.com/ee/ci/environments/protected_environments.html)
-
-***
-
```

```
-## Description
-Let people know what your project can do specifically. Provide context and add a link to any reference visitors might be unfamiliar with. A list of Features or a Background subsection can also be added here. If there are alternatives to your project, this is a good place to list differentiating factors.
-
-## Badges
-On some READMEs, you may see small images that convey metadata, such as whether or not all the tests are passing for the project. You can use Shields to add some to your README. Many services also have instructions for adding a badge.
-
-## Visuals
-Depending on what you are making, it can be a good idea to include screenshots or even a video (you'll frequently see GIFs rather than actual videos). Tools like ttygif can help, but check out Asciinema for a more sophisticated method.
-
-## Installation
-Within a particular ecosystem, there may be a common way of installing things, such as using Yarn, NuGet, or Homebrew. However, consider the possibility that whoever is reading your README is a novice and would like more guidance. Listing specific steps helps remove ambiguity and gets people to using your project as quickly as possible. If it only runs in a specific context like a particular programming language version or operating system or has dependencies that have to be installed manually, also add a Requirements subsection.
-
-## Usage
-Use examples liberally, and show the expected output if you can. It's helpful to have inline the smallest example of usage that you can demonstrate, while providing links to more sophisticated examples if they are too long to reasonably include in the README.
-
-## Support
-Tell people where they can go to for help. It can be any combination of an issue tracker, a chat room, an email address, etc.
-
```

```
-## Contributing
-State if you are open to contributions and what your requirements are for accepting them.
-
-For people who want to make changes to your project, it's helpful to have some documentation on how to get started. Perhaps there is a script that they should run or some environment variables that they need to set. Make these steps explicit. These instructions could also be useful to your future self.
-
-You can also document commands to lint the code or run tests. These steps help to ensure high code quality and reduce the likelihood that the changes inadvertently break something. Having instructions for running tests is especially helpful if it requires external setup, such as starting a Selenium server for testing in a browser.
-
-## Authors and acknowledgment
-Show your appreciation to those who have contributed to the project.
-
-## License
-For open source projects, say how it is licensed.
-
-## Project status
-If you have run out of energy or time for your project, put a note at the top of the README saying that development has slowed down or stopped completely. Someone may choose to fork your project or volunteer to step in as a maintainer or owner, allowing your project to keep going. You can also make an explicit request for maintainers.
-change on develop
+*   **Issues:** Track tasks, enhancements, and bugs under **Issues** in GitLab.
+*   **Merge Requests (MRs):** Review and merge code changes. Link MRs to issues.
+*   **Wiki:** Project documentation can be maintained in the GitLab Wiki.
+*   **Settings:** Configure integrations, webhooks, and access permissions under **Settings**.
+*   **Members:** Manage team access under **Project Information > Members**.
```

In this way, in my workflow, LLM first generated the improvement (*README*), then the pipeline automatically created a branch, pushed the changes, and automatically opened the MR, which is an advanced and practical application of LLM in DevOps automation.

References:

[1] GitLab, ".gitlab-ci.yml reference," *GitLab Documentation.* Available: https://docs.gitlab.com/ee/ci/yaml/

[2] GitLab, "Stages," *GitLab Documentation.* Available: https://docs.gitlab.com/ee/ci/yaml/#stages

[3] GitLab, "Artifacts," *GitLab Documentation.* Available: https://docs.gitlab.com/ee/ci/yaml/#artifacts

[4] GitLab, "when (job execution policy: on_success/on_failure/always)," *GitLab Documentation.* Available: https://docs.gitlab.com/ee/ci/yaml/#when

[5] GitLab, "rules," GitLab Documentation. Available: https://docs.gitlab.com/ee/ci/yaml/#rules

[6] GitLab, "tags (select runners for jobs)," *GitLab Documentation.* Available: https://docs.gitlab.com/ee/ci/yaml/#tags

[7] GitLab, "CI/CD variables," *GitLab Documentation*. Available: https://docs.gitlab.com/ee/ci/variables/

[8] GitLab, "Create a merge request (API)," *GitLab REST API Documentation.* Available: https://docs.gitlab.com/ee/api/merge_requests.html#create-a-new-merge-request

[9] PyCQA, "Bandit: a security linter for Python," *Bandit Documentation.* Available: https://github.com/PyCQA/bandit

[10] Python Software Foundation, "venv - Creation of virtual environments," *Python Documentation.* Available: https://docs.python.org/3/library/venv.html

[11] Python Enhancement Proposals, "PEP 668 - Marking Python base environments as 'externally managed'," *PEP Index.* Available: https://peps.python.org/pep-0668/

[12] Kubernetes, "Deployments," *Kubernetes Documentation.* Available: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[13] Kubernetes, "Service (including NodePort)," *Kubernetes Documentation.* Available: https://kubernetes.io/docs/concepts/services-networking/service/

[14] Kubernetes, "kubectl apply," *Kubernetes Reference Documentation.* Available: https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#apply

[15] minikube (Kubernetes SIGs), “minikube service,” *minikube Documentation.* Available: https://minikube.sigs.k8s.io/docs/commands/service/

[16] DeepSeek, “API Documentation,” *DeepSeek API Docs.* Available: https://api-docs.deepseek.com/

[17] Python Software Foundation, “Requests: HTTP for Humans,” *Python Package Index (PyPI).* Available: https://pypi.org/project/requests/

[18] Ollama, “Ollama (install/run models),” *GitHub Repository.* Available: https://github.com/ollama/ollama

[19] Microsoft, “Phi-3 Mini 4K Instruct (ONNX) model card,” *Hugging Face.* Available: https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-onnx

[20] Python Packaging Authority, “pip list,” *pip Documentation.* Available: https://pip.pypa.io/en/stable/cli/pip_list/

[21] Trail of Bits, “pip-audit,” GitHub Repository. Available: https://github.com/pypa/pip-audit

[22] Aqua Security, “Trivy: Container image vulnerability scanner,” *Trivy Documentation*. Available: https://aquasecurity.github.io/trivy/